UNIT-5 Interrupt

Dr. Suresh C
EEE DEPT
RVCE

Interrupts

- Motivations
  - Inform a program of some external events timely
    - Polling *vs* Interrupt
  - Implement multi-tasking with priority support

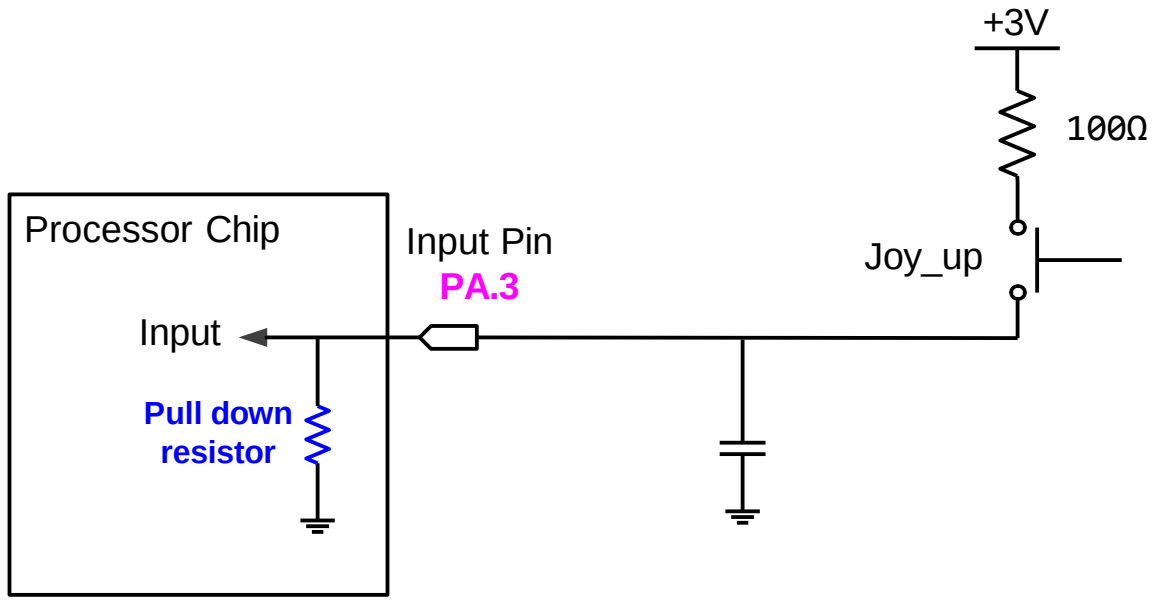
Polling *vs* Interrupt

Polling:
You pick up the phone every three seconds to check whether you are getting a call.

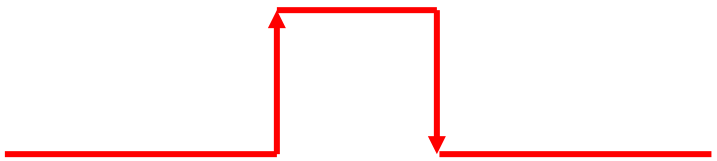
Interrupt:
Do whatever you should do and pick up the phone when it rings.

Example: Push a button to turn on a LED

- Check whether a button has been pressed?
- **Polling**
  - Repeatedly read IDR and check whether bit 3 is set (i.e., busy wait)
  - OK if CPU has nothing else to do
- **Interrupt**
  - When hardware detects a rising or fall edge, hardware generates a service request
  - CPU responses to the service request and starts to execute the corresponding service subroutine

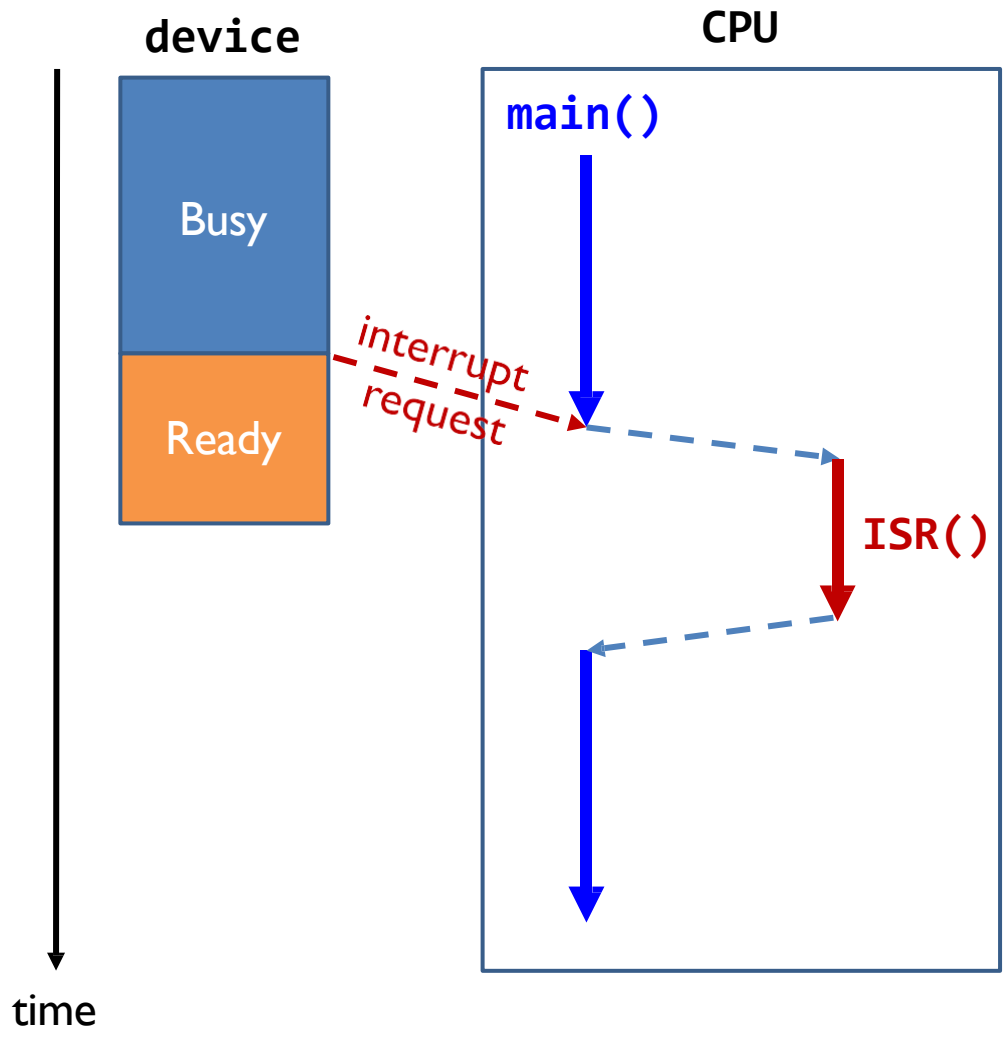


Voltage on PA.3



Polling *vs* Interrupt

- Interrupt-driven operations
  - Allows CPU to perform other tasks until external/internal devices require service
  - CPU automatically stops the current code and starts to execute an interrupt handler (or called interrupt service routine, **ISR**)

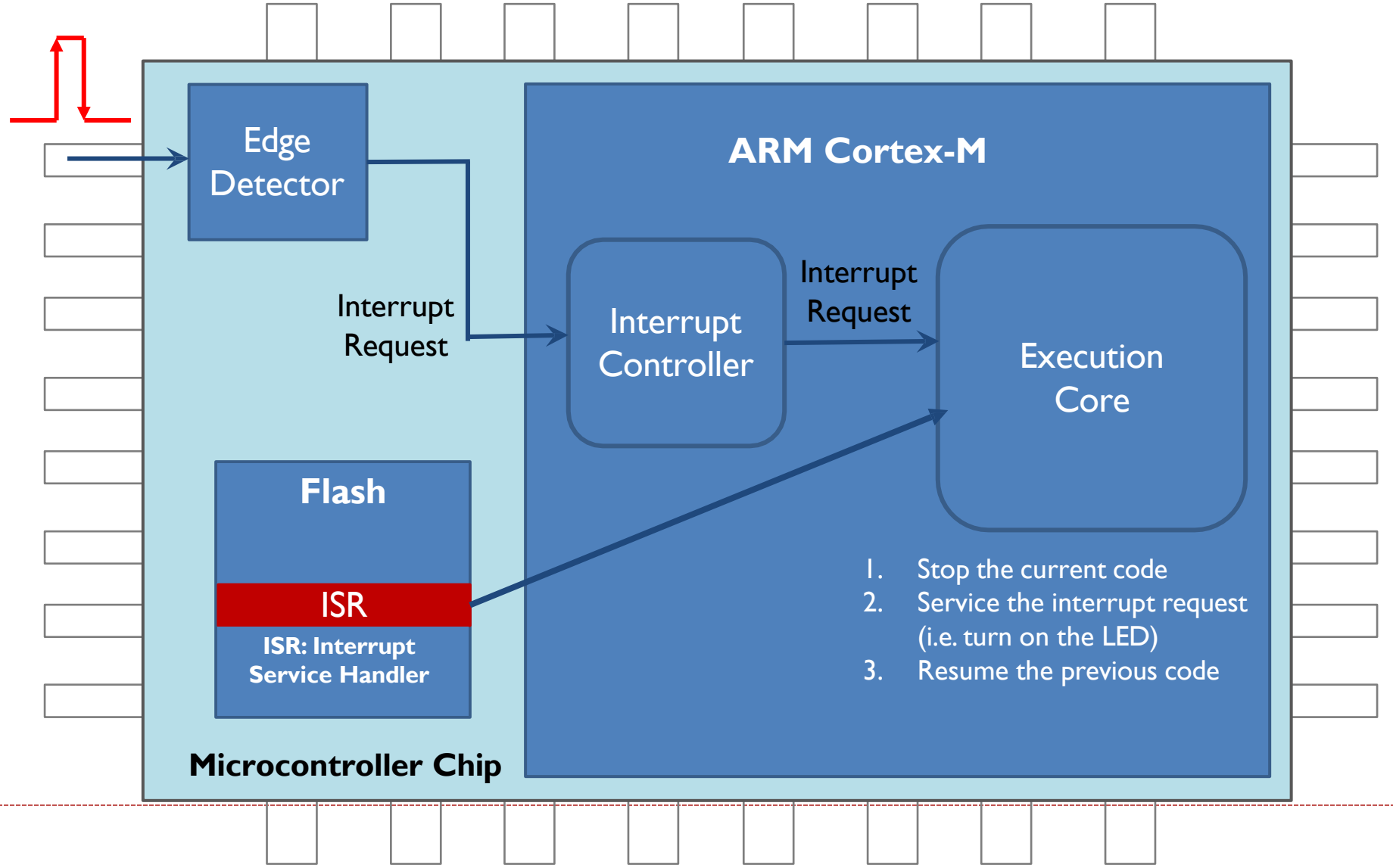


Polling *vs* Interrupt

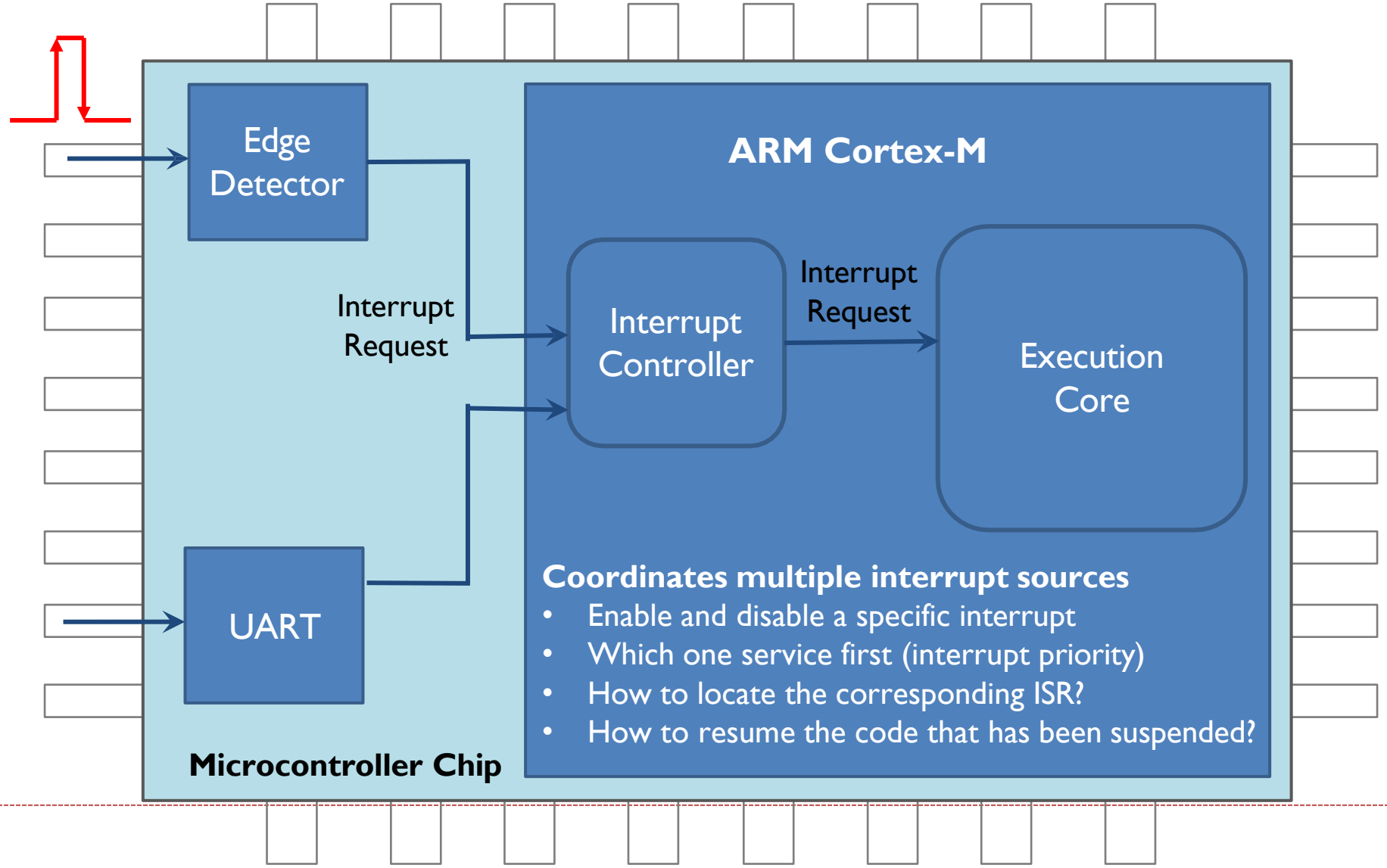
- Interrupt-driven operations
  - Allows CPU to perform other tasks until external/internal devices require service
  - CPU automatically stops the current code and starts to execute an interrupt handler (or interrupt service routine)

Polling	Interrupt
Software periodically checks	CPU only takes actions only if an event occurs
Waste lot of CPU cycles	Does not waste much CPU cycle
Triggered by software	Triggered by hardware or software
Occurs periodically	Occurs at any time

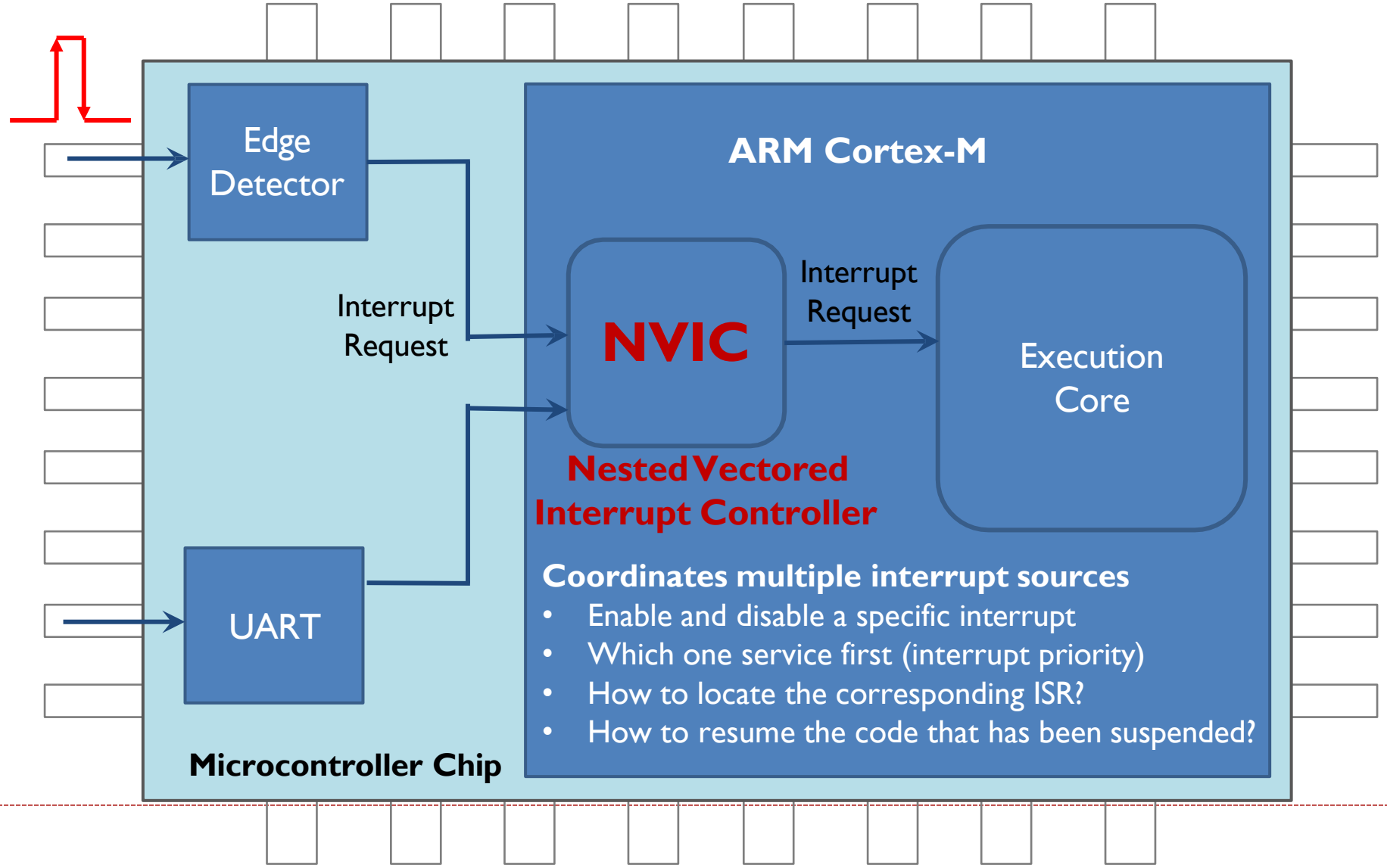
How to support interrupt?



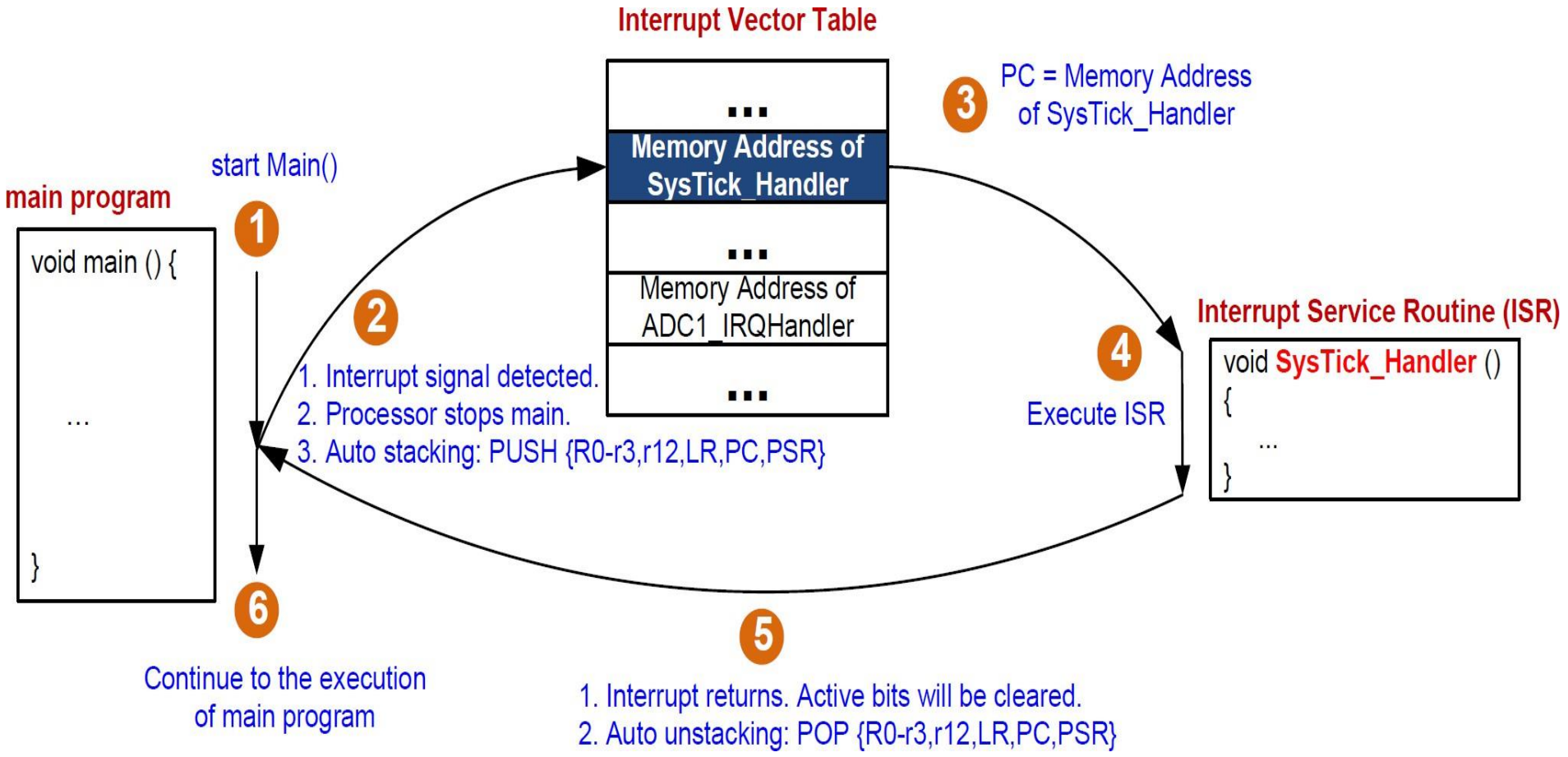
How to support interrupt?



How to support interrupt?



Interrupt


Interrupt Vector Table
...
Memory Address of SysTick_Handler
...
Memory Address of ADC1_IRQHandler
...
3
PC = Memory Address of SysTick_Handler
start Main()
main program
1
void main () {
...
}
2
1. Interrupt signal detected.
2. Processor stops main.
3. Auto stacking: PUSH {R0-r3,r12,LR,PC,PSR}
Interrupt Service Routine (ISR)
4
Execute ISR
void SysTick_Handler ()
{
...
}
6
Continue to the execution of main program
5
1. Interrupt returns. Active bits will be cleared.
2. Auto unstacking: POP {R0-r3,r12,LR,PC,PSR}

Interrupt Service Routine Vector Table

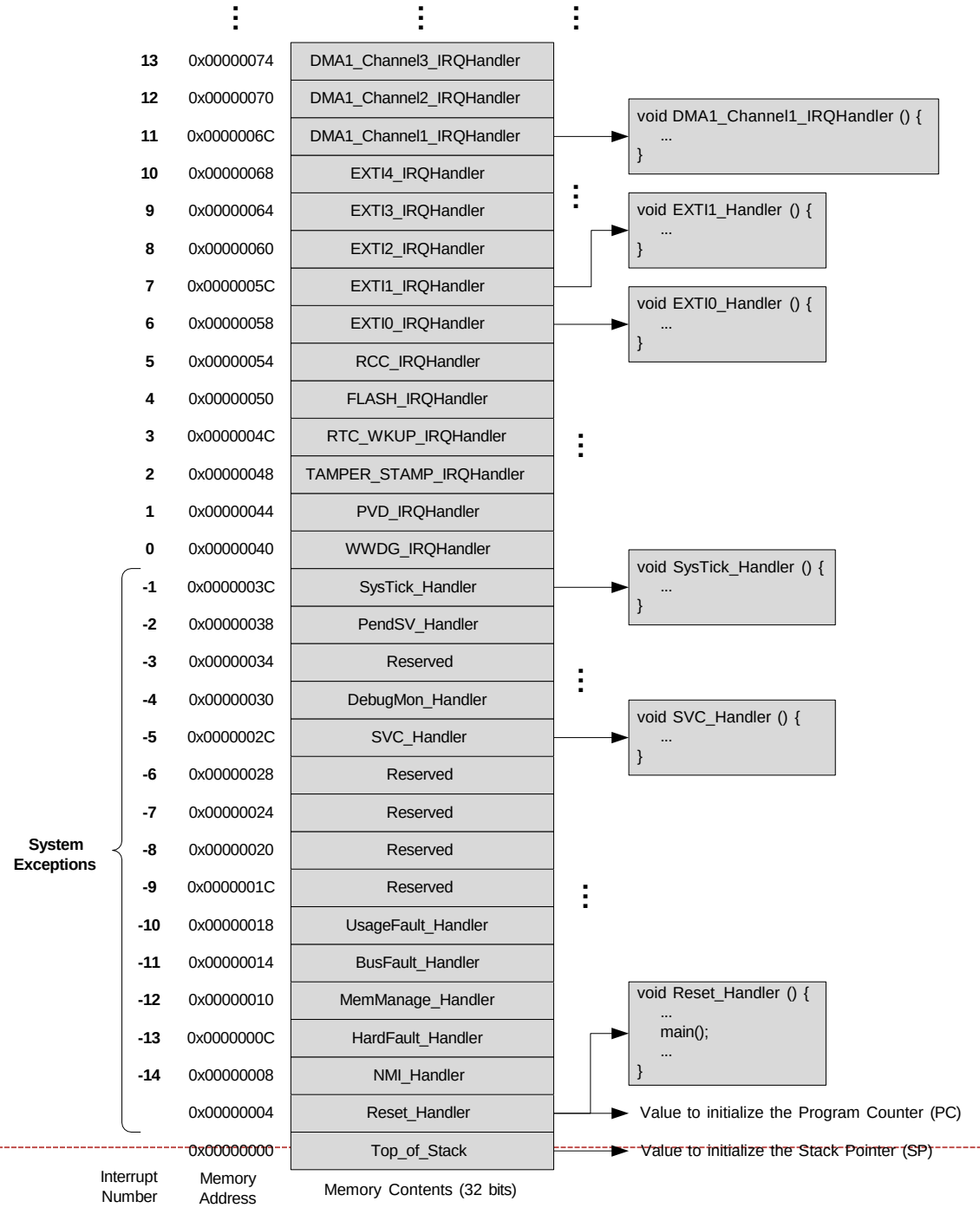
- Start address for the exception hander for each exception type is fixed and pre-defined
- Processor loads PC with this fixed, pre-defined address
- Exception Vector Table starts at memory address 0
- Program Counter pc = 0x00000004 initially

Address	Priority	Type of priority	Acronym	Description
0x0000_0000	-	-	-	Stack Pointer
0x0000_0004	-3	fixed	Reset	Reset Vector
0x0000_0008	-2	fixed	NMI_Handler	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.
0x0000_000C	-1	fixed	HardFault_Handler	All class of fault
0x0000_0010	0	settable	MemManage_Handler	Memory management
0x0000_0014	1	settable	BusFault_Handler	Pre-fetch fault, memory access fault
0x0000_0018	2	settable	UsageFault_Handler	Undefined instruction or illegal state
0x0000_001C-0x0000_002B	-	-	-	Reserved
0x0000_002C	3	settable	SVC_Handler	System service call via SWI instruction
0x0000_0030	4	settable	DebugMon_Handler	Debug Monitor
0x0000_0034	-	-	-	Reserved
0x0000_0038	5	settable	PendSV_Handler	Pendable request for system service
0x0000_003C	6	settable	SysTick_Handler	System tick timer
...				

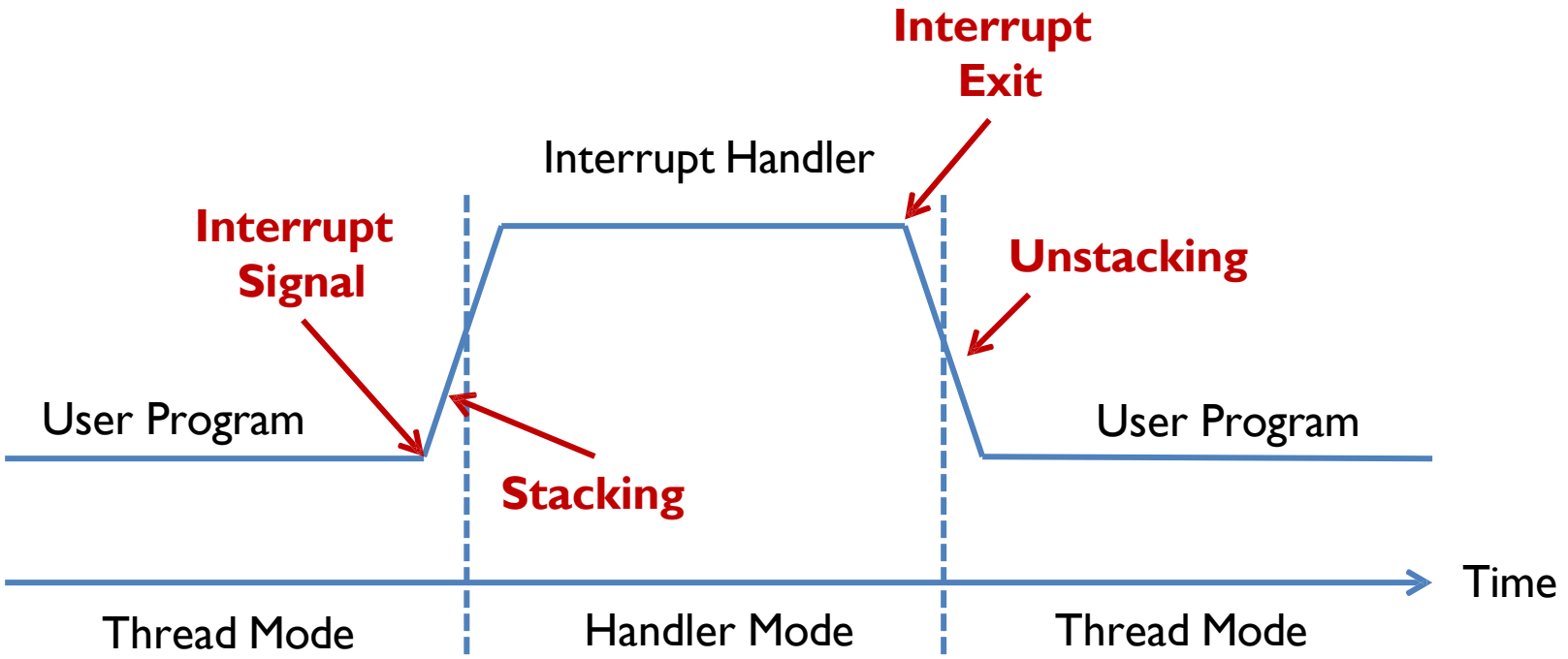
ISR Vector Table

For interrupt number **n**: (the interrupt shown in the xPSR)

CMSIS Interrupt Number = **16 + n**



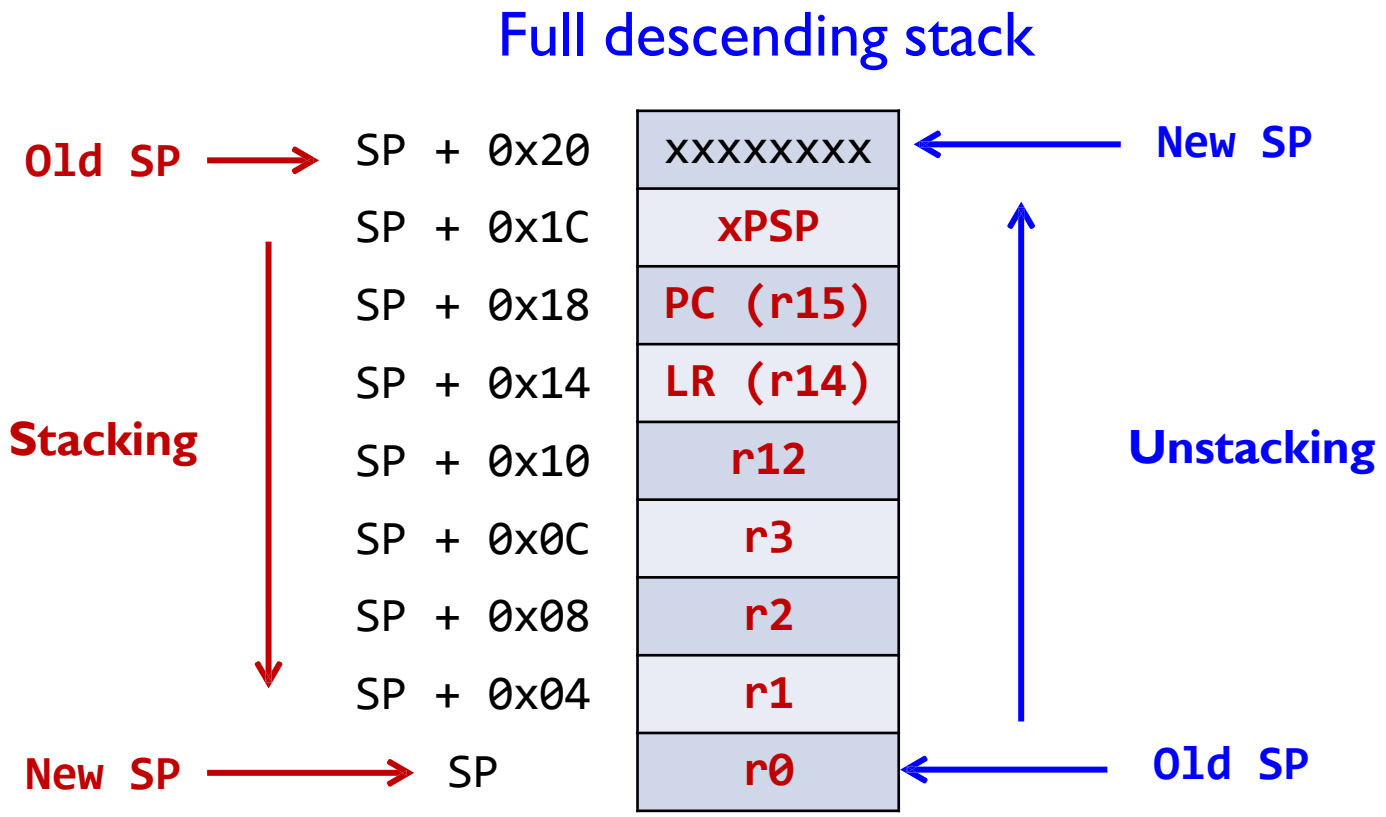
Automatic Stacking & Unstacking



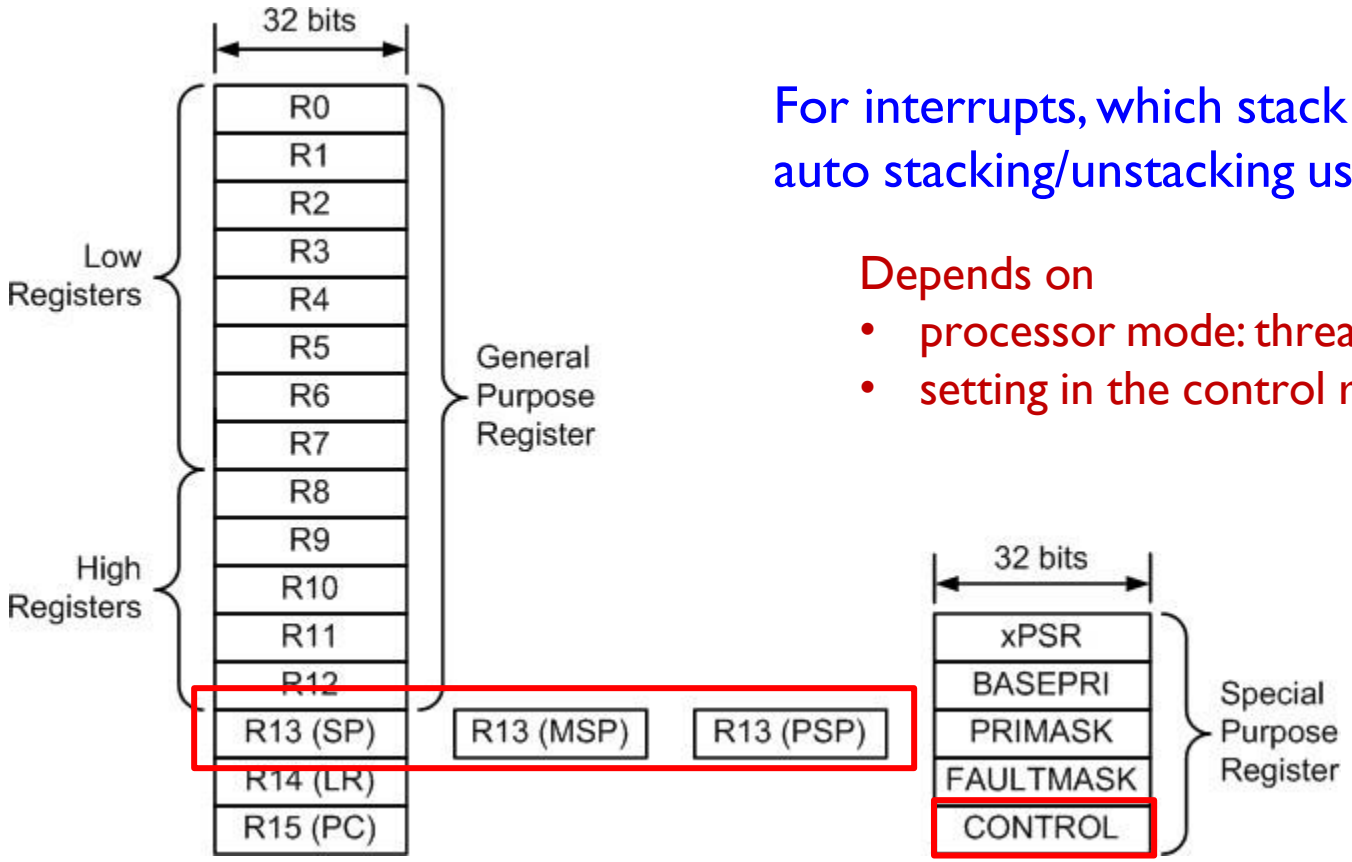
Stacking: hardware automatically pushes eight register into the stack
(`xPSR,PC,LR,r12,r3,r2,r1,r0`)

Unstacking: hardware automatically pops these eight register off the stack

Automatic Stacking & Unstacking



Stack Pointer: MSP *vs* PSP



For interrupts, which stack does auto stacking/unstacking use?

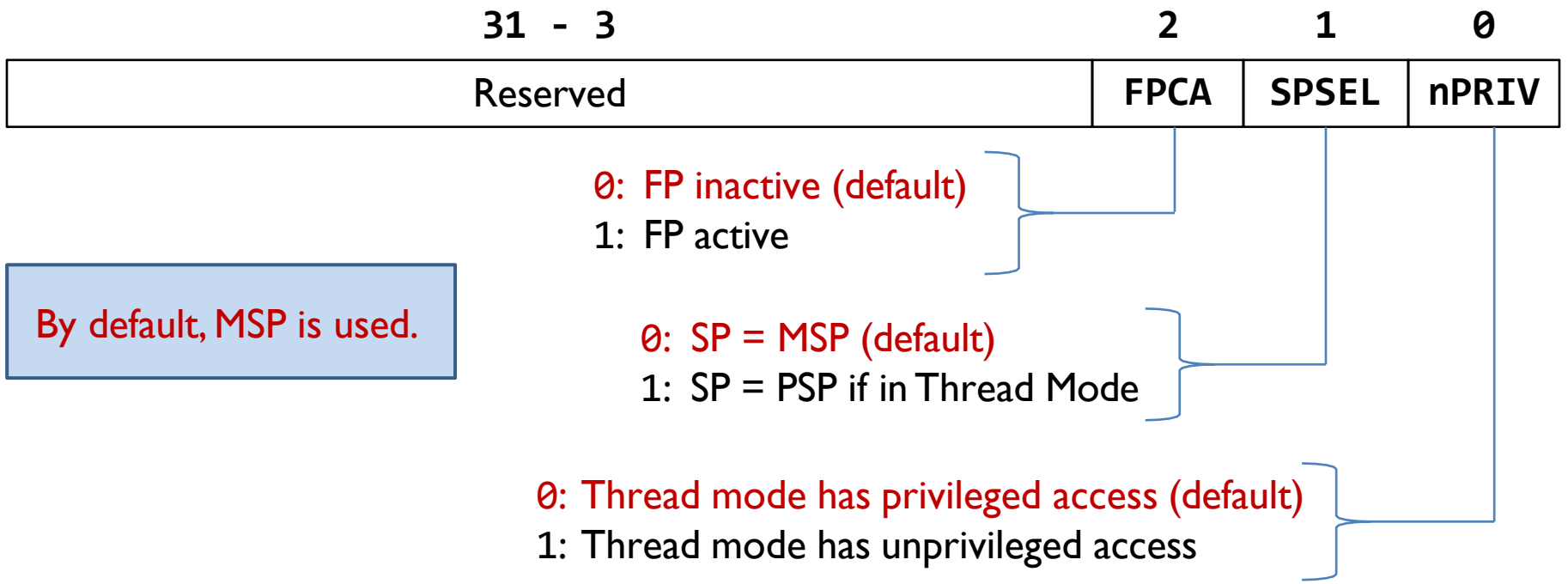
Depends on

- processor mode: thread vs handler
- setting in the control register

- **MSP**: Main Stack Pointer (selected at reset)
- **PSP**: Process Stack Pointer
- In assembly, both MSP and PSP are called R13 or SP

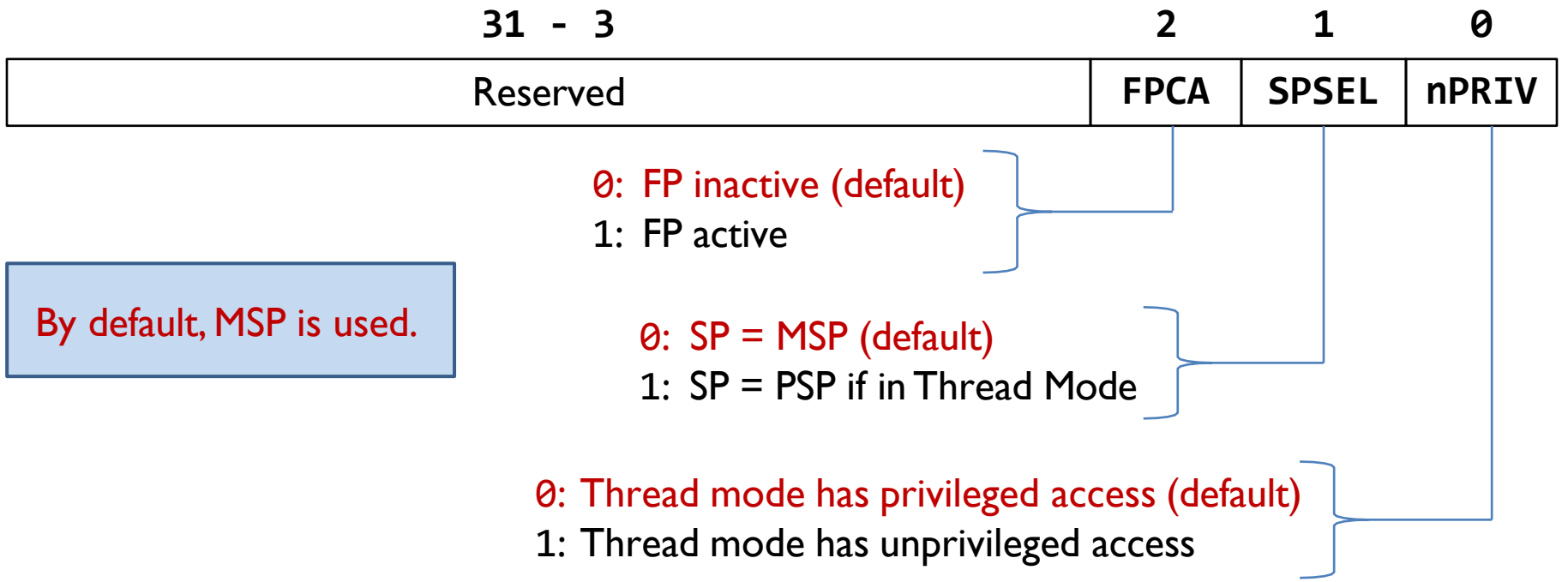
no	Thread Mode	Handler Mode
1	Also known as **"Thread"** or **"User Mode."**	Also referred to as **"Handler"** or **"Privileged Mode."**
2	This is the normal execution mode for an application's main code.	Handler Mode is used to execute exception and interrupt service routines (**ISRs**).
3	In Thread Mode, the processor executes application code, and the thread can be preempted by higher-priority exceptions, such as interrupts.	When an exception occurs, the processor transitions to Handler Mode to execute the corresponding exception handler.
4	When an exception occurs, the processor saves the current context (registers and program counter) and transfers control to the appropriate exception handler.	In Handler Mode, all interrupts and exceptions are treated as non-maskable, and they can preempt the execution of lower-priority exception handlers or thread code.
5	Multiple exceptions can be pending, and the processor services them in priority order. However, lower-priority exceptions can be delayed if higher-priority exceptions are being handled.	Handler Mode has higher privileges and access to privileged system resources, while Thread Mode is more restricted in terms of system access.
6	Thread Mode is used for executing non-critical code and application tasks.	Handler Mode is used for executing critical system tasks, such as interrupt service routines and operating system kernels.

Control Register




```
; Access special registers
MRS  r0, CONTROL    ; Read CONTROL into R0
ORRS r0, R0, #0x2   ; Set SPSEL to select PSP
MSR  CONTROL, r0    ; Write R0 into CONTROL
ISB                 ; Instruction Synchronization Barrier
```

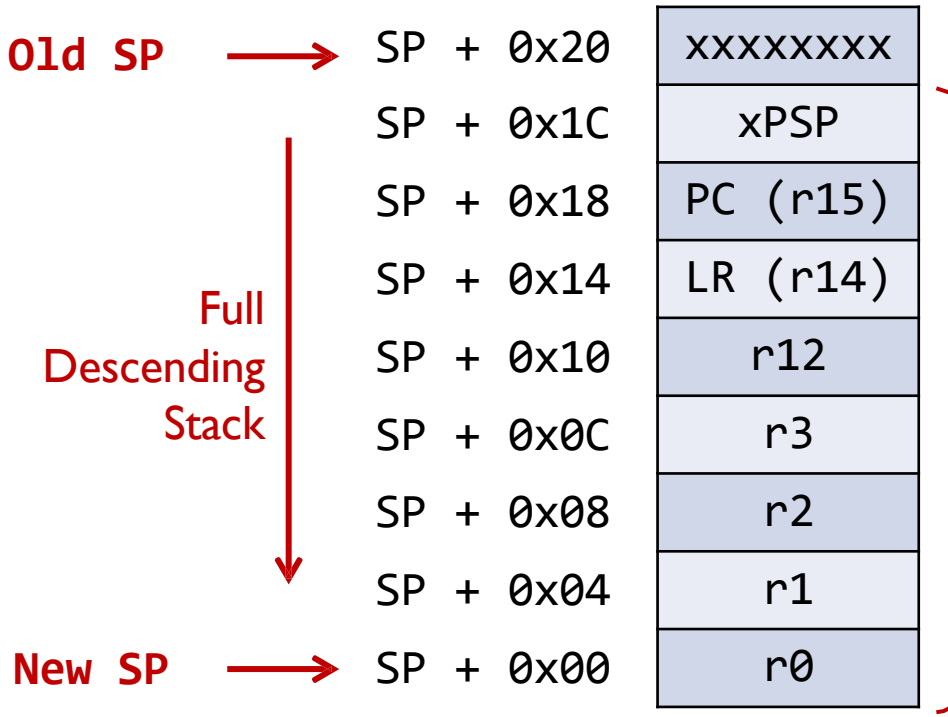
Control Register



	SP	Privileged
Handler Mode	**SP = MSP** and **SPSEL** = 0	Privileged
Thread Mode	Depending on **SPSEL**	Depending on **nPRIV**

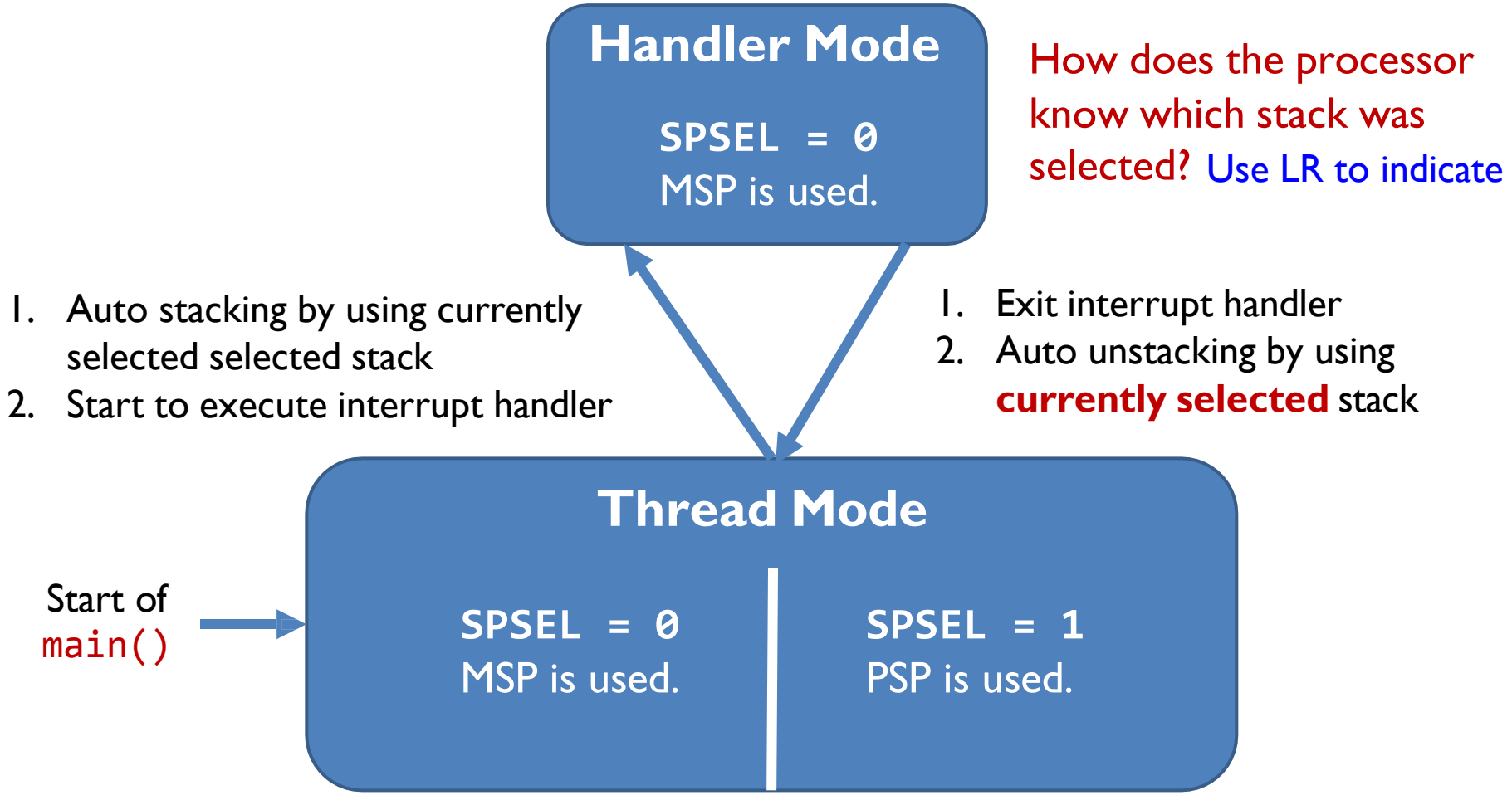
For simple applications, MSP is used.

Automatic Stacking & Unstacking

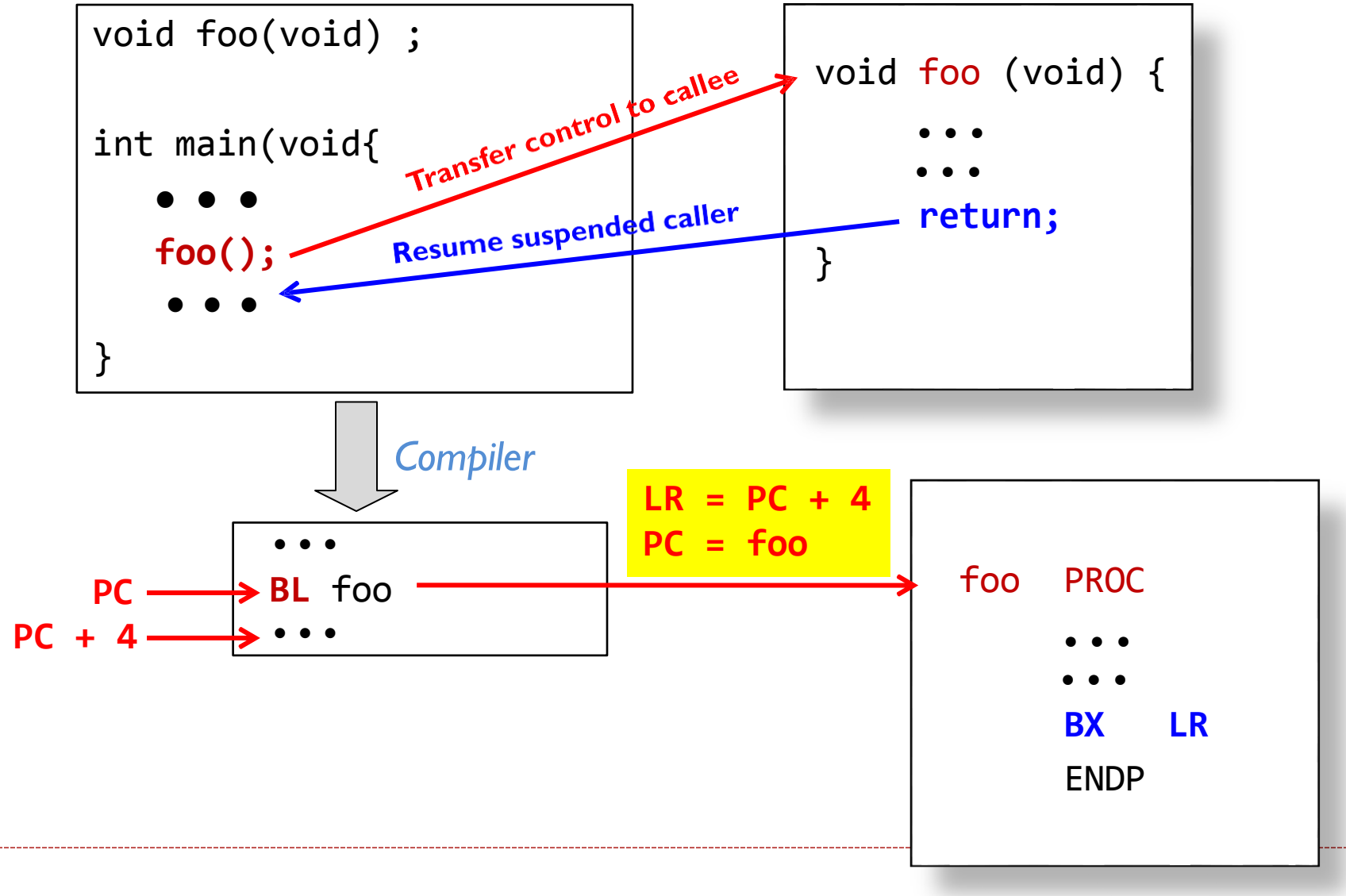


- **Stacking**: The processor automatically pushes these eight registers into the **currently selected stack** before an interrupt handler starts
- **Unstacking**: The processor automatically pops these eight register out of the **currently selected stack** when an interrupt hander exits.

MSP *vs* PSP


Handler Mode
SPSEL = 0
MSP is used.
How does the processor know which stack was selected? Use LR to indicate
1. Auto stacking by using currently selected selected stack
2. Start to execute interrupt handler
1. Exit interrupt handler
2. Auto unstacking by using currently selected stack
Thread Mode
Start of main()
SPSEL = 0
MSP is used.
SPSEL = 1
PSP is used.

Recall: Link Register for calling functions



Link Register (LR)

- When software calls a subroutine, LR holds the returning address.
- Does does LR hold when serving an interrupt?

Which stack to use when an interrupt returns?

Link Register (LR) now has two usages:

- LR = address of the instruction immediately after BL
- LR indicates whether MSP or PSP is used to restore register from when exiting an interrupt

FPU was not used before interrupt (FPCA bit in `Control` register = `0`):

Return Mode	Link Register (LR)	Return Stack
Handler	`0xFFFFFFF1`	`SP = MSP`
Thread	`0xFFFFFFF9`	`SP = MSP`
	`0xFFFFFFFD`	`SP = PSP`

FPU was used before interrupt (FPCA bit in `Control` register = `1`):

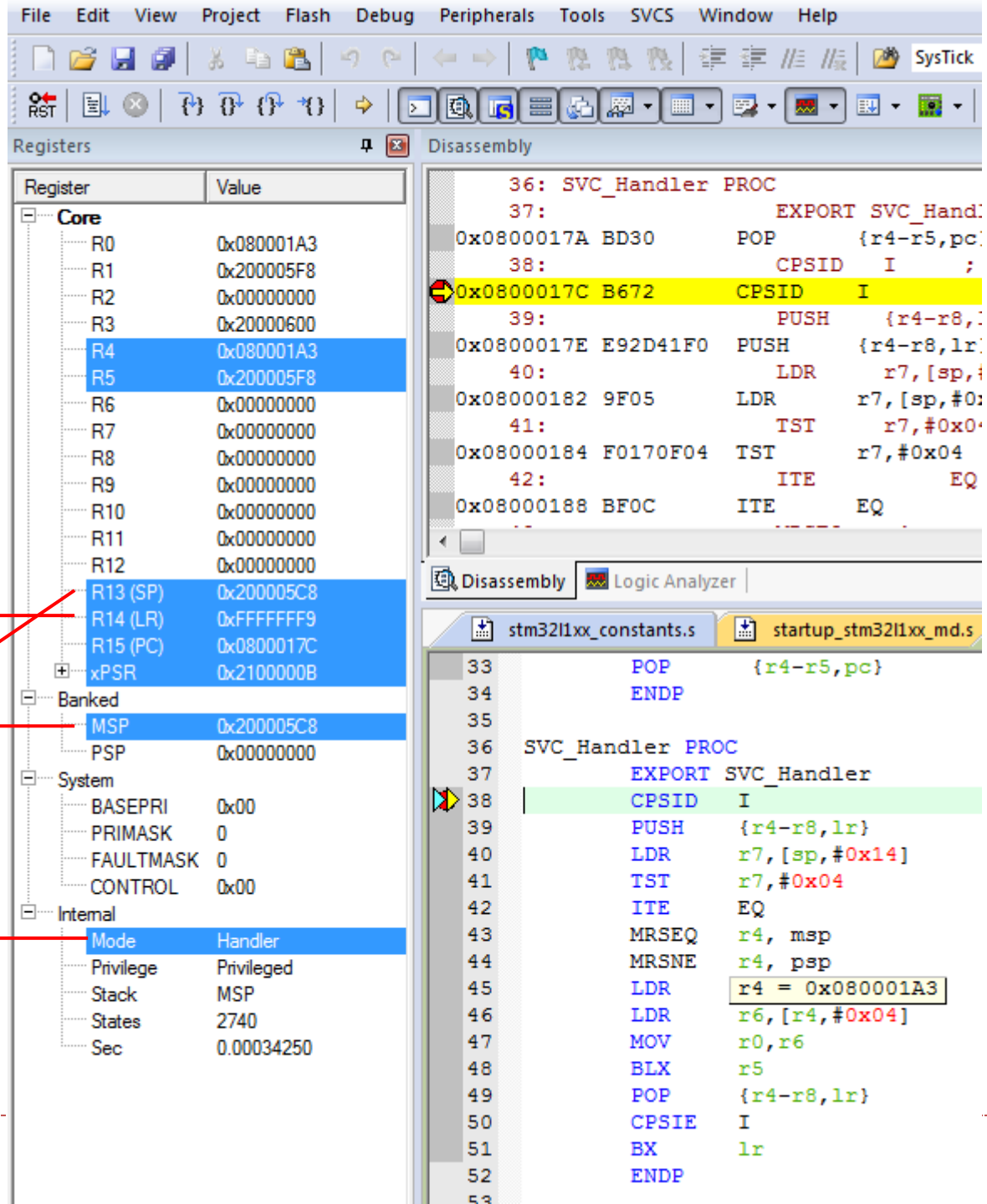
Return Mode	Link Register (LR)	Return Stack
Handler	`0xFFFFFFE1`	`SP = MSP`
Thread	`0xFFFFFFE9`	`SP = MSP`
	`0xFFFFFFED`	`SP = PSP`

Register values in an interrupt service routine

LR = 0xFFFFFFF9

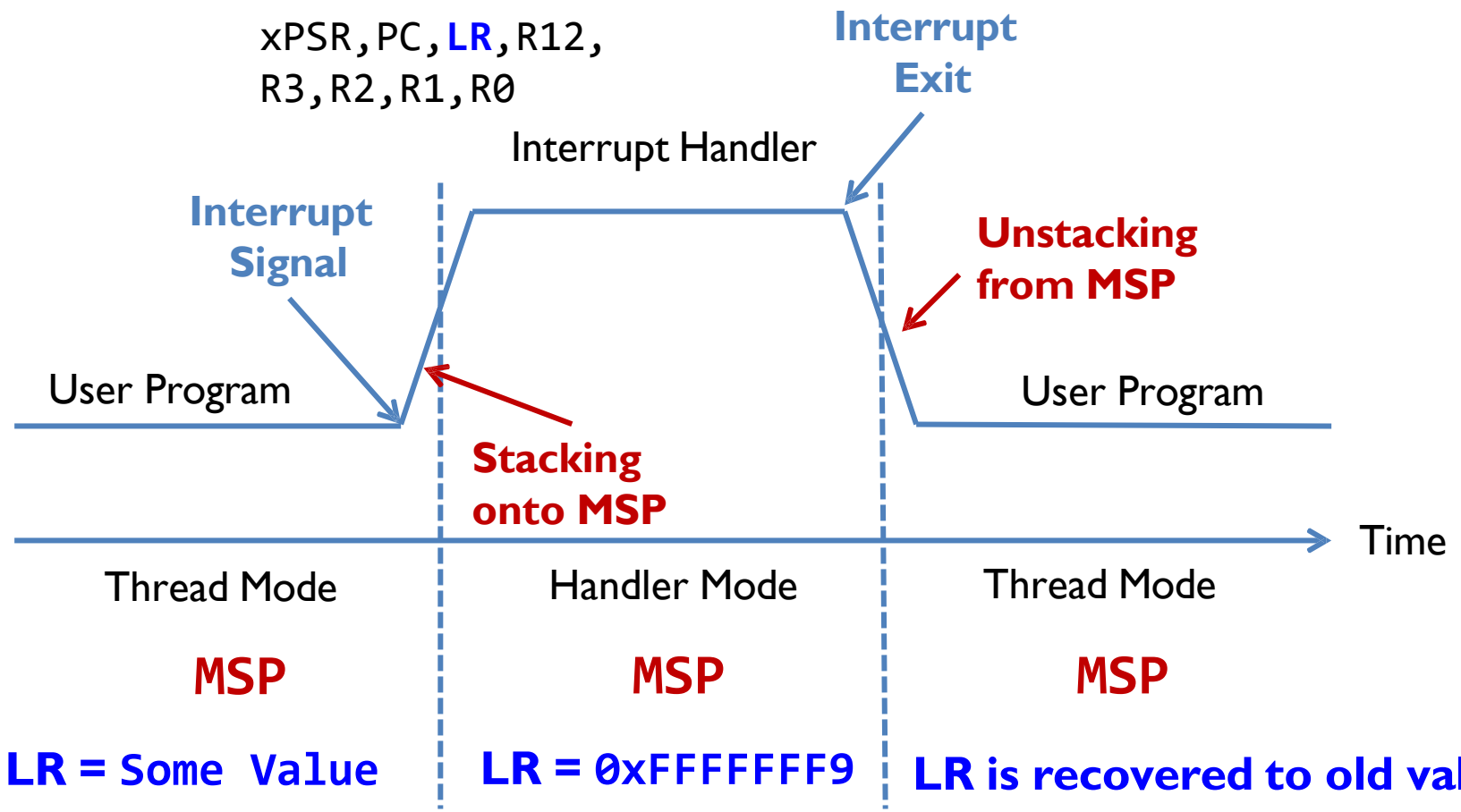
SP = MSP

ISR always in hander mode.



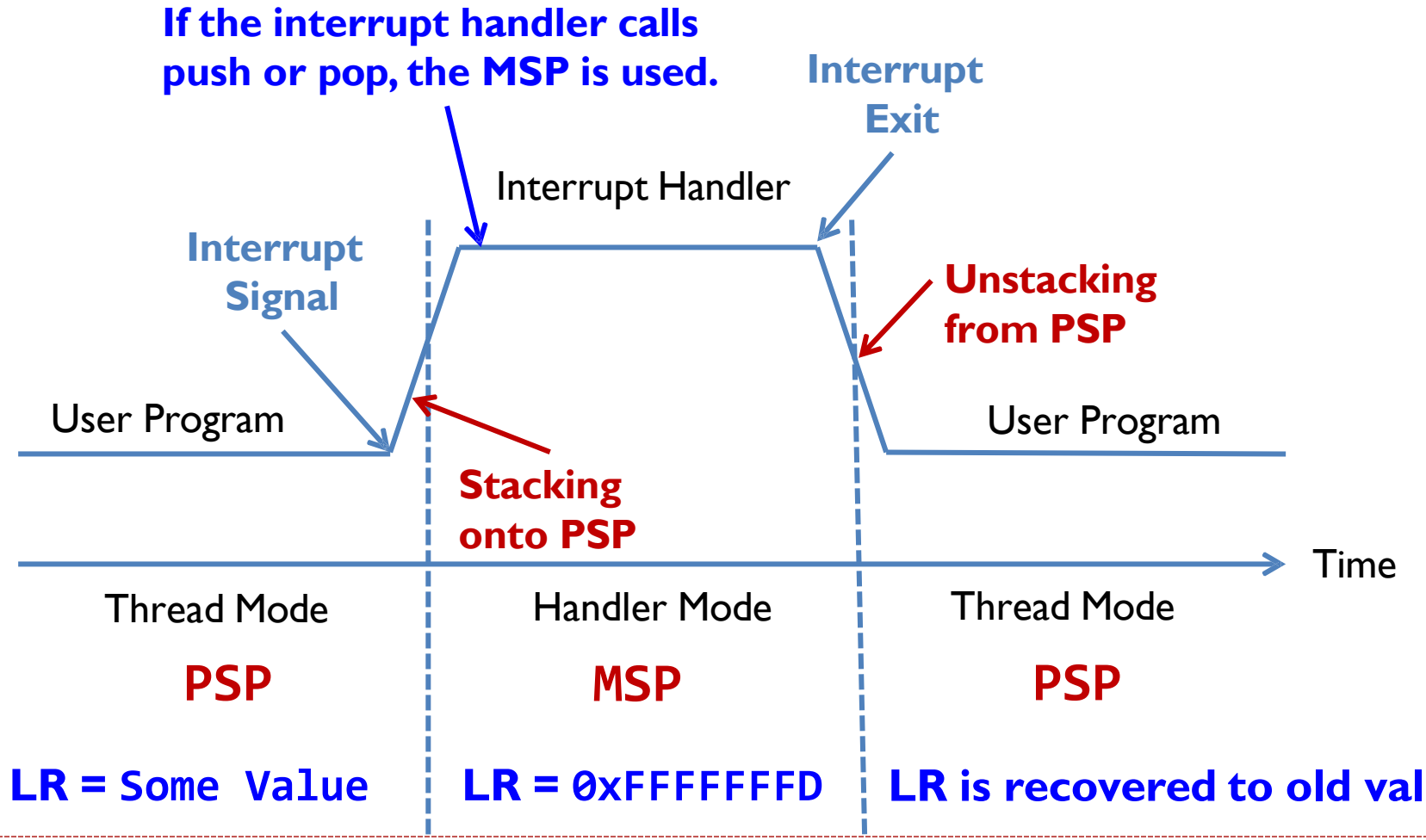
Stacking & Unstacking

Assume SPSEL = 0 and no FP is used ⟹ User program uses MSP.



Stacking & Unstacking

Assume SPSEL = 1 and no FP is used ⟹ User program uses PSP.



Which stack was used for stacking?

- Suppose in an interrupt handler, we want to push registers onto the same stack that was selected for automatic stacking
- After an interrupt:
  - If LR = 0xFFFFFFF**9**, then MSP was used for auto stacking
  - If LR = 0xFFFFFFF**D**, then PSP was used for auto stacking

9 = 1001

D = 1101

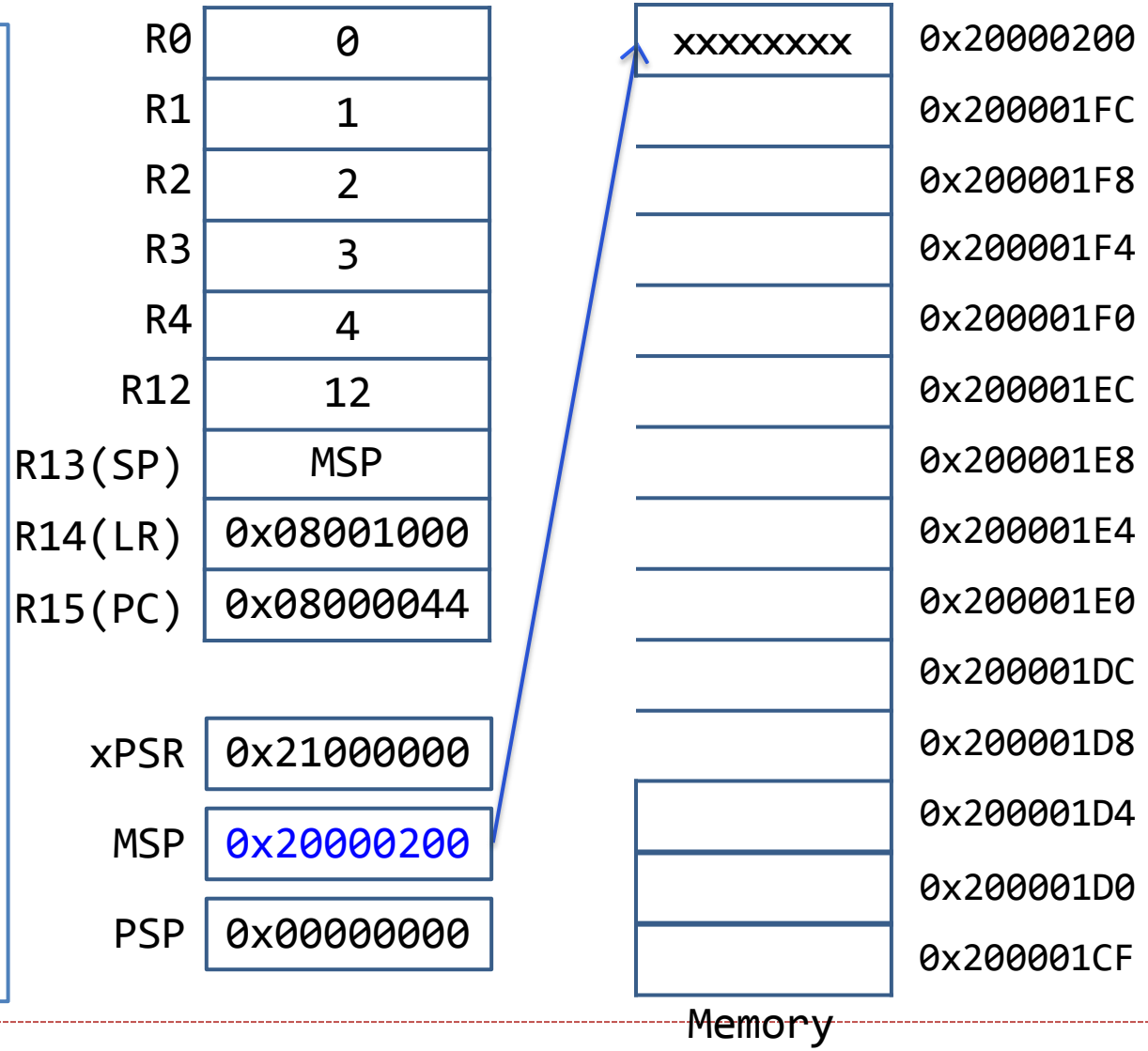
```
SysTick_Handler
  EXPORT SysTick_Handler

  TST   LR, #4          ; check bit 2 of LR
  MRSEQ r0, msp         ; if 0, MSP was used
  MRSNE r0, psp         ; if 1, PSP was used
  STMFD r0!, {r7-r9}    ; Push onto a Full Descending Stack
  ; Note: we cannot use PUSH {r7-r9} because PUSH uses MSP
  ; in an interrupt service routine
```

Interrupt: Stacking & Unstacking

```
__main PROC
   …
   MOV r3,#0
   …
   ENDP

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

addr = 0x08000044

addr = 0x0800001C



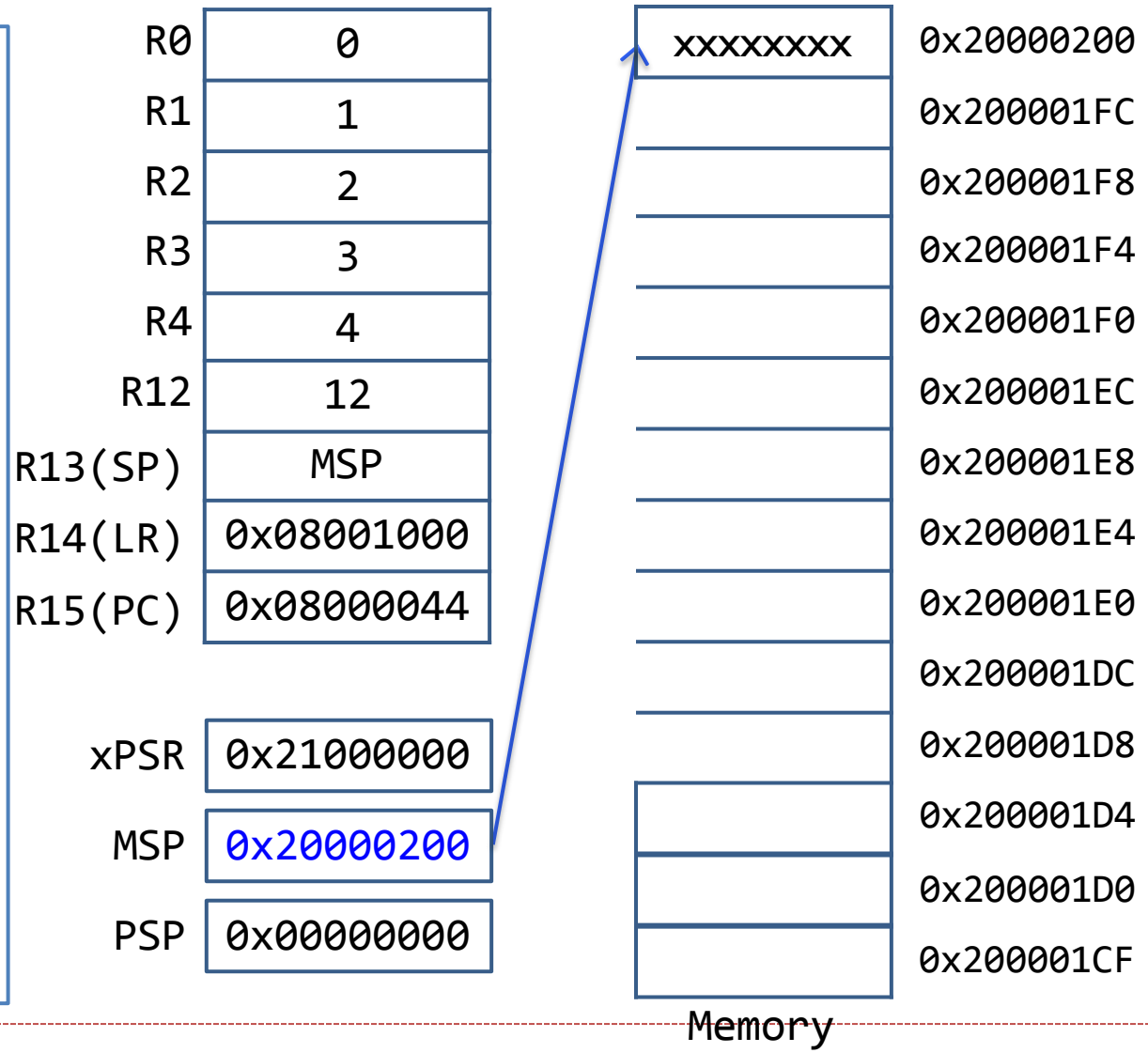
Interrupt:

Suppose SysTick interrupt occurs when PC = 0x08000044

```
__main PROC
   …
   MOV r3,#0
   …
   ENDP

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

addr = 0x08000044

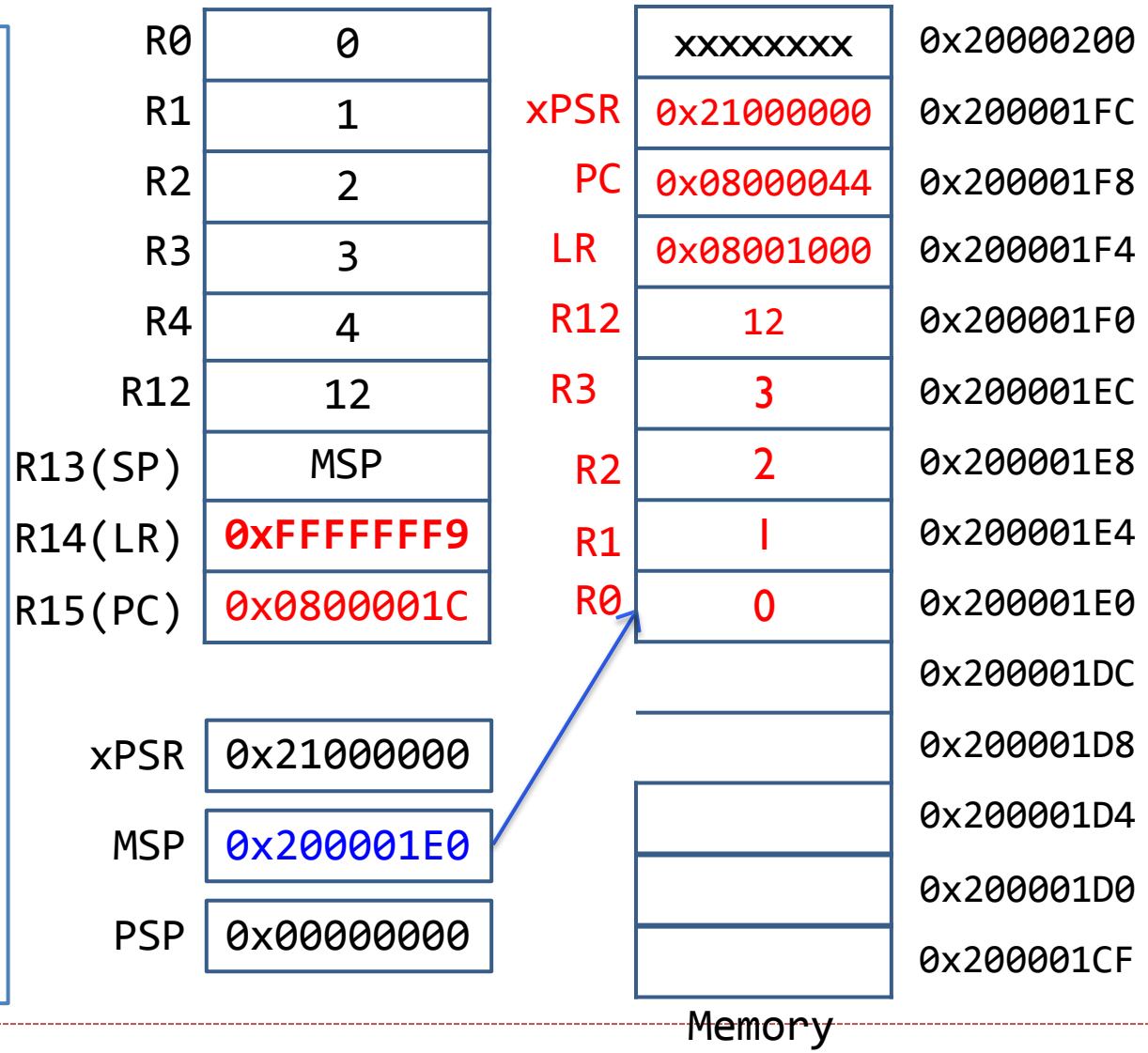
addr = 0x0800001C



Interrupt: Stacking & Uns

STACKING

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

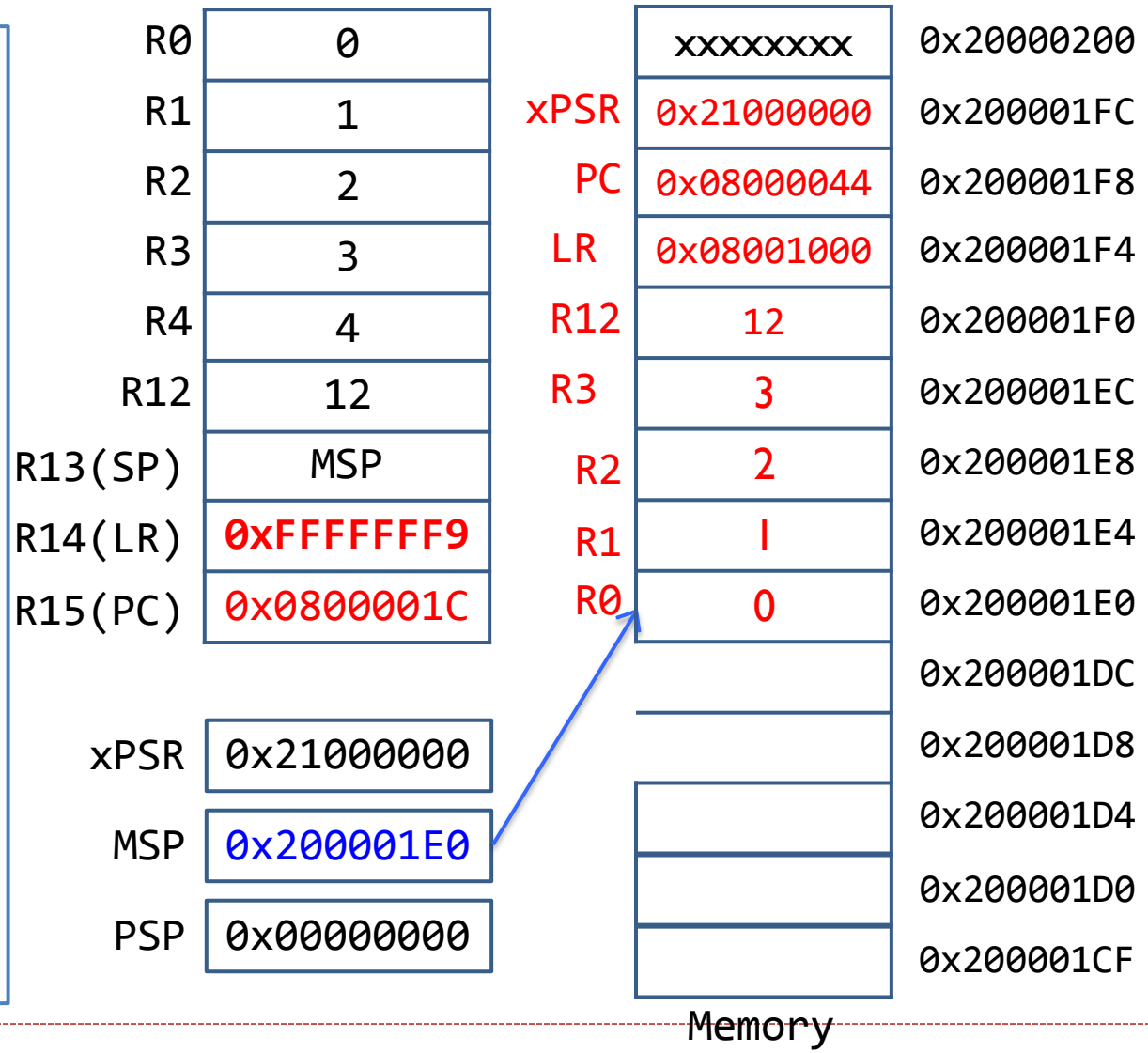


Interrupt: Stacking & Uns

STACKING

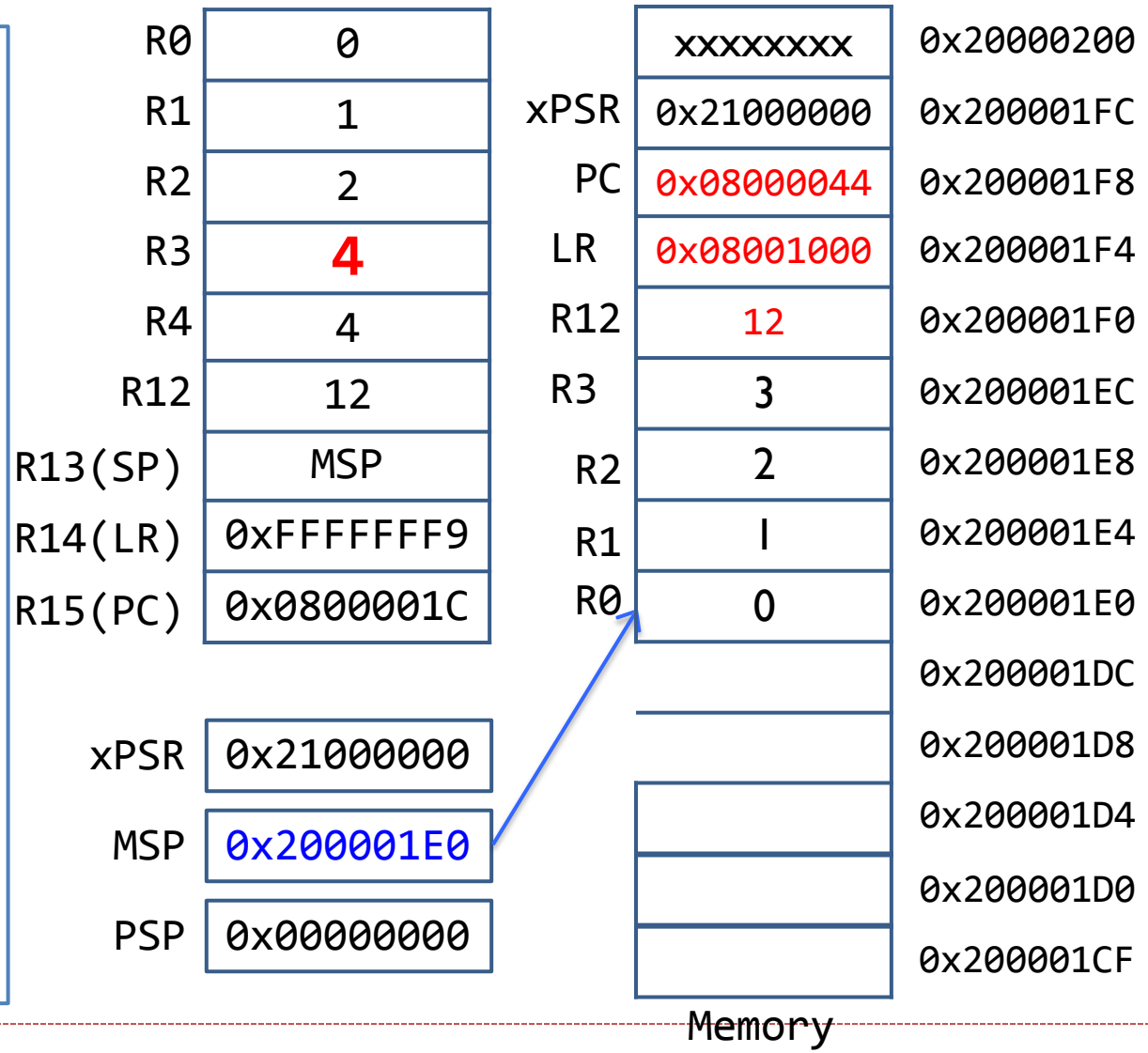
```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

LR = 0xFFFFFFF9 to indicate MSP is used.



Interrupt: Stacking & Unstacking

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```



Interrupt: Stacking & Unstacking

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

R0	0
R1	1
R2	2
R3	4
R4	5
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFF9
R15(PC)	0x08000020

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

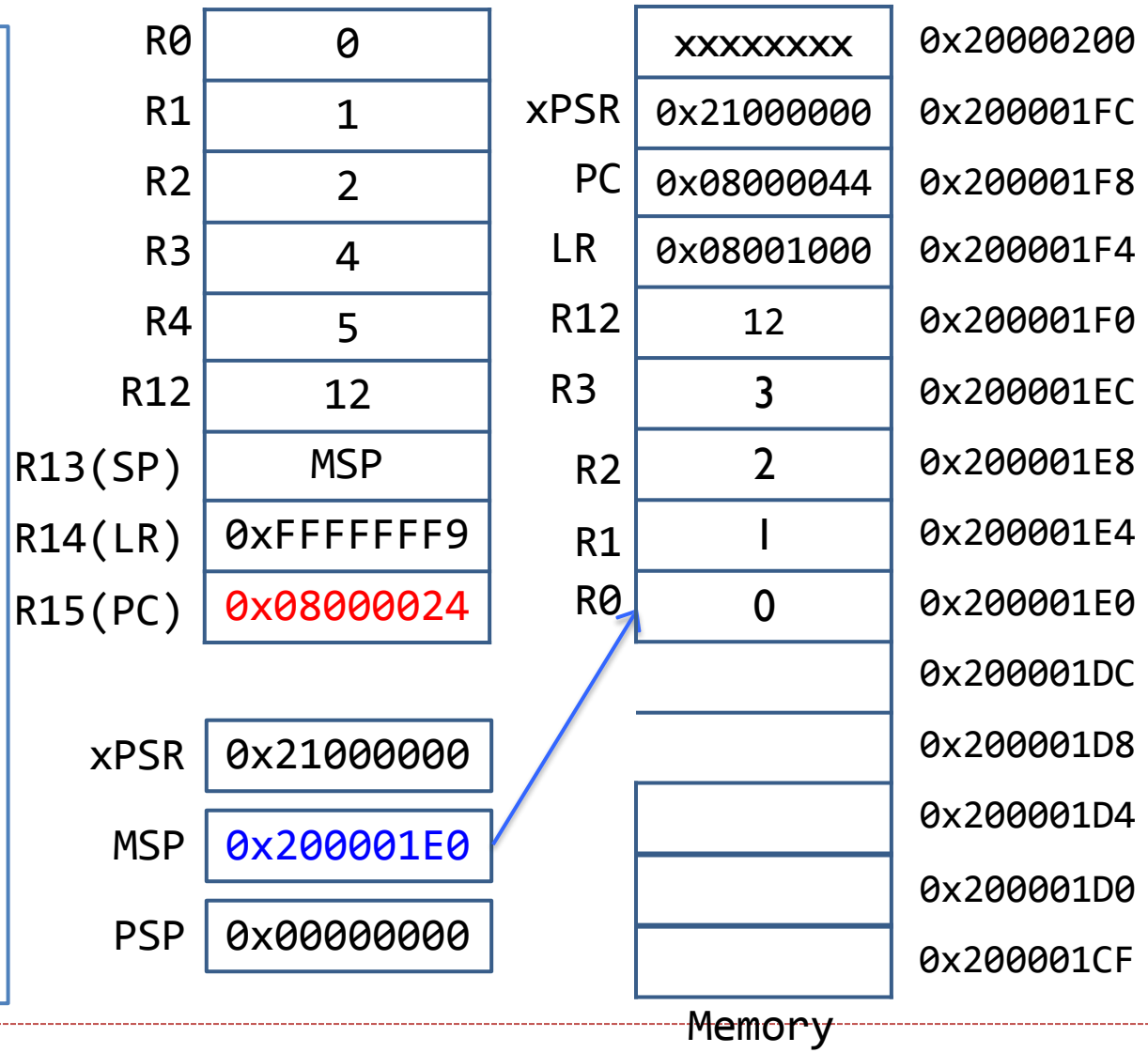
	xxxxxxxx	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
LR	0x08001000	0x200001F4
R12	12	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Interrupt: Stacking & Unstacking

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

LR = 0xFFFFFFF9 to indicate MSP is used.



Interrupt: Stacking & Uns UNSTACKING

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

LR = 0xFFFFFFF9 to indicate MSP is used.

R0	0
R1	1
R2	2
R3	4
R4	5
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFF9
R15(PC)	0x08000024

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

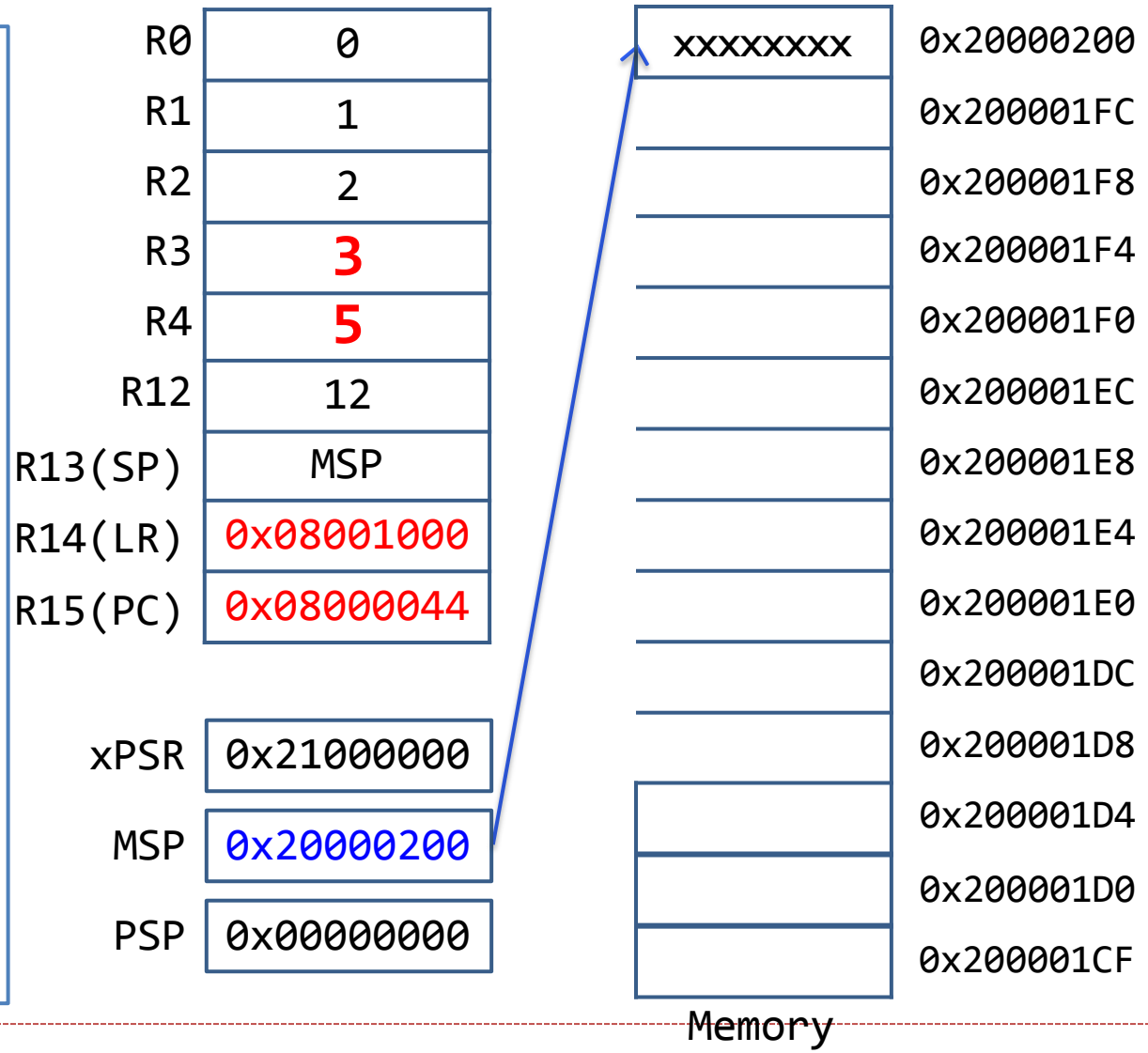
	xxxxxxxx	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
LR	0x08001000	0x200001F4
R12	12	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Interrupt: Stacking & Uns UNSTACKING

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

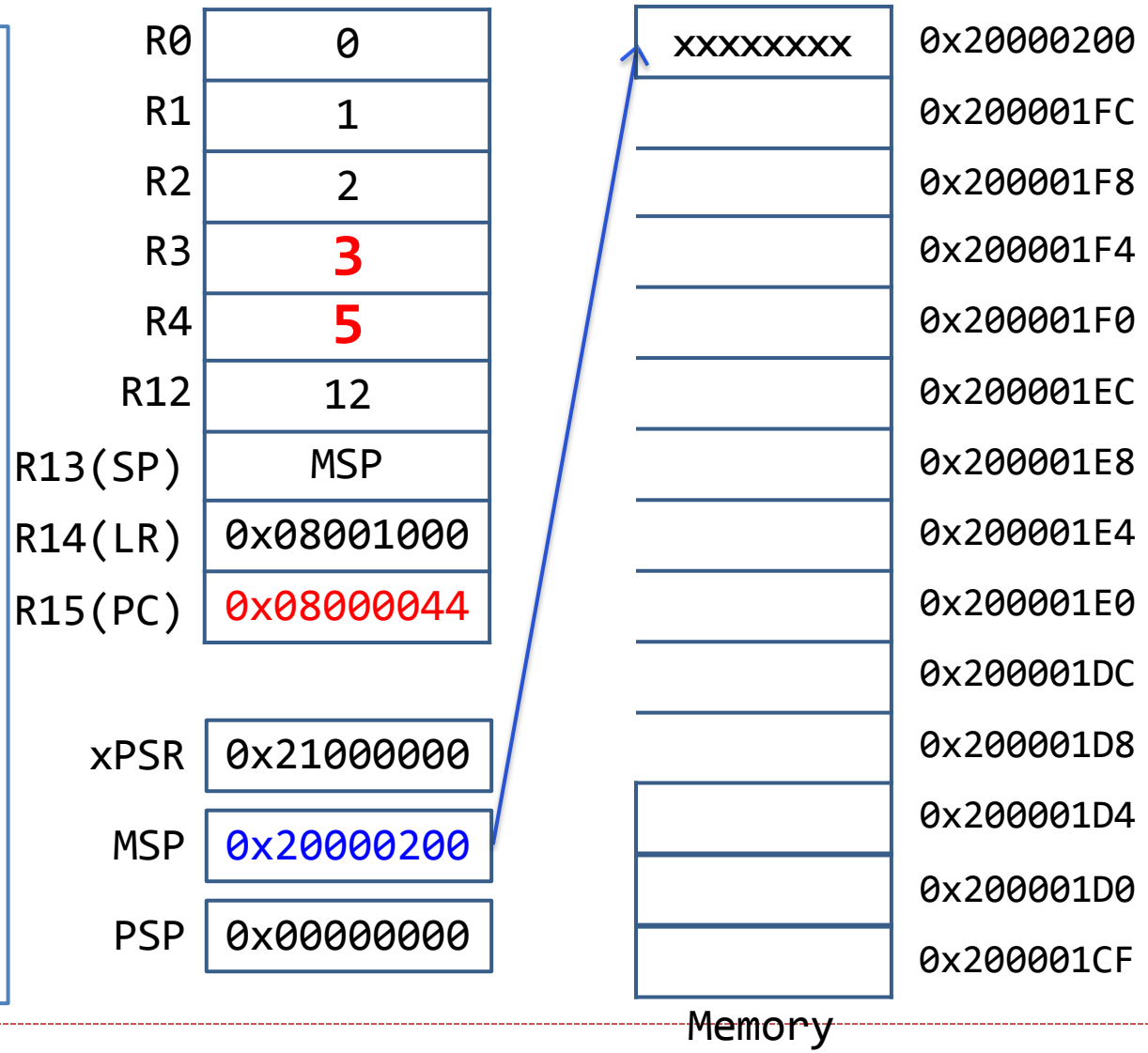
Note the new value of R3 is lost!!!



Interrupt: Stacking & Unstacking

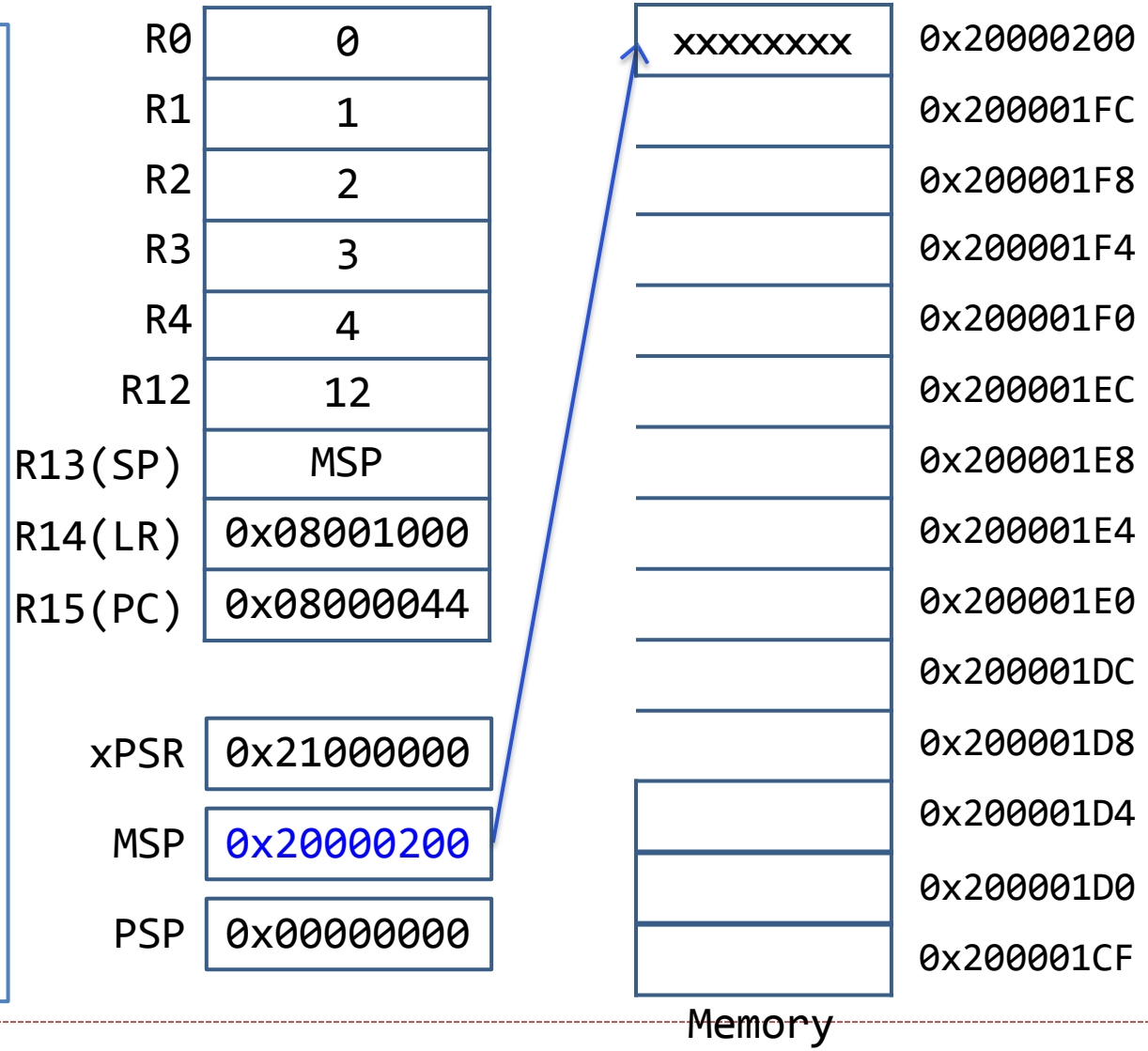
```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

The Main program resumes!!!



Interrupt: Stacking & Unstacking

```
__main PROC
                addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP
                addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1 0x0800001C
   BL  sine   0x08000020
   BX  lr     0x08000024
   ENDP
```



Interrupt: Stacking & Uns

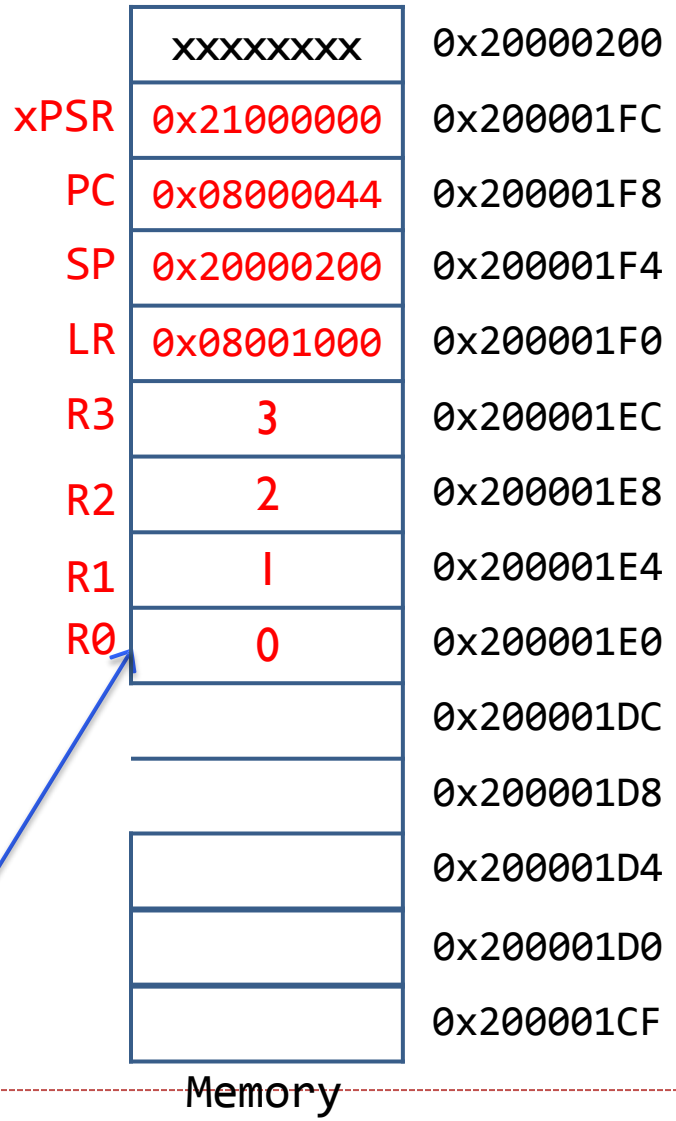
STACKING

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1   0x0800001C
   BL  sine     0x08000020
   BX  lr       0x08000024
   ENDP
```

LR = 0xFFFFFFF9 to indicate MSP is used.

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFF9
R15(PC)	0x0800001C

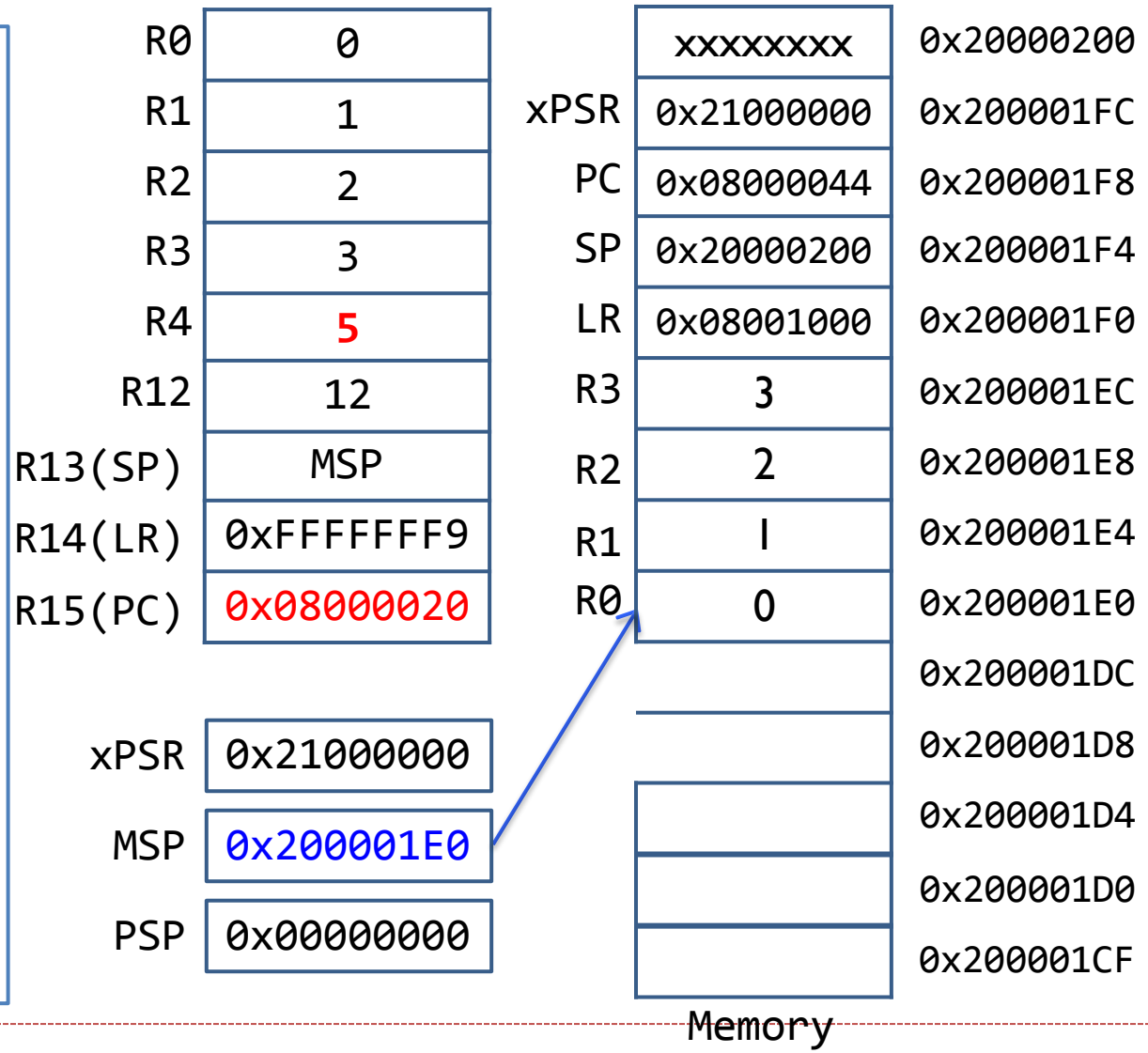
xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000



Interrupt: Stacking & Unstacking

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1 0x0800001C
   BL  sine   0x08000020
   BX  lr     0x08000024
   ENDP
```

LR = 0xFFFFFFF9 to indicate MSP is used.



Interrupt: Stacking & Unstacking

```
__main PROC
   …
   MOV r3,#0
   …
   ENDP

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1 0x0800001C
   BL  sine   0x08000020
   BX  lr     0x08000024
   ENDP
```

addr = 0x08000044

addr = 0x0800001C

BL sine
Updates LR register

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0x08000024
R15(PC)	0x080000F0

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	xxxxxxxx	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
SP	0x20000200	0x200001F4
LR	0x08001000	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

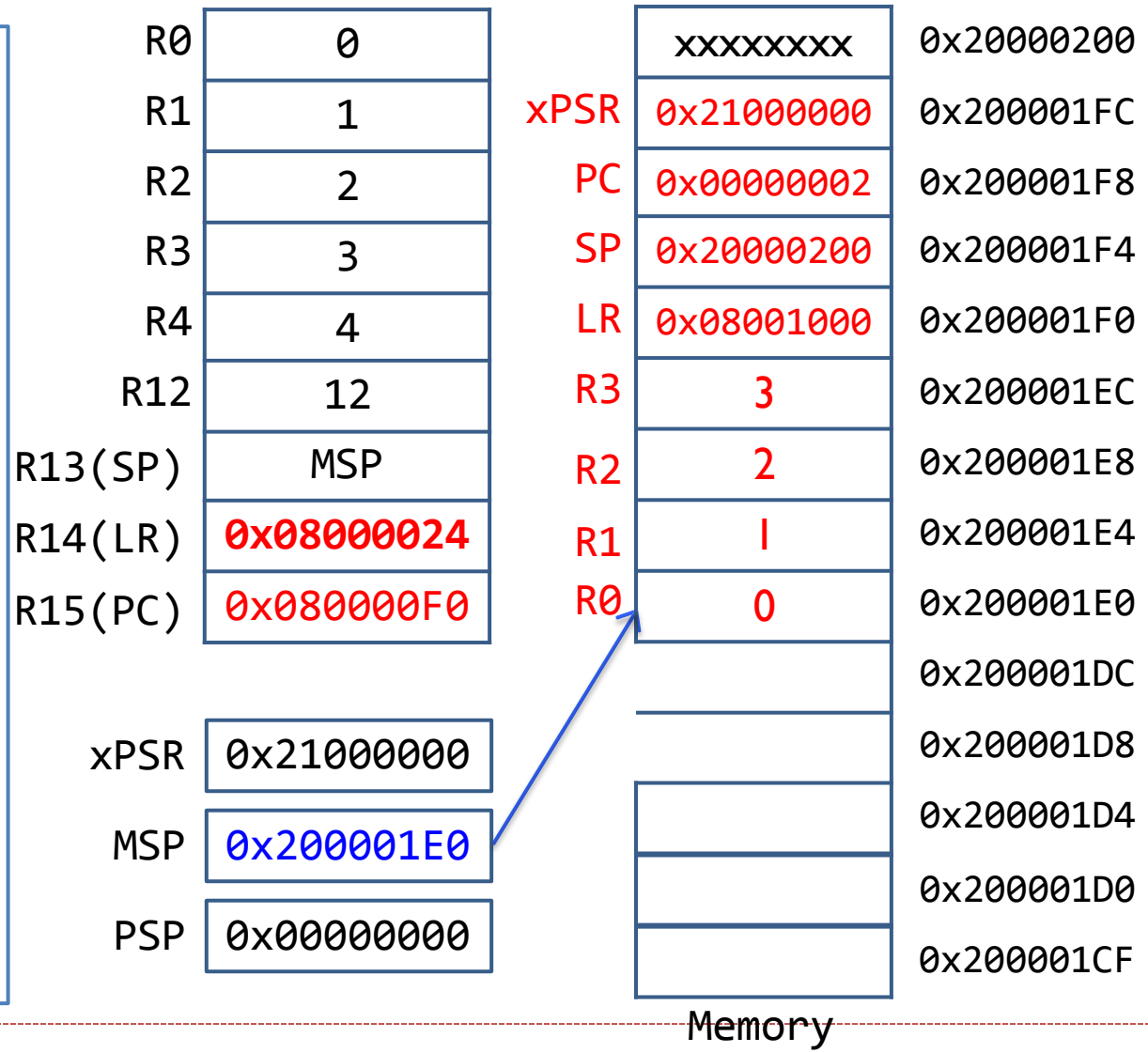
Memory

Assume sine() is located at 0x08000024.

Interrupt: Stacking & Unstacking

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1 0x0800001C
   BL  sine   0x08000020
   BX  lr     0x08000024
   ENDP
```

BL sine
Updates LR register

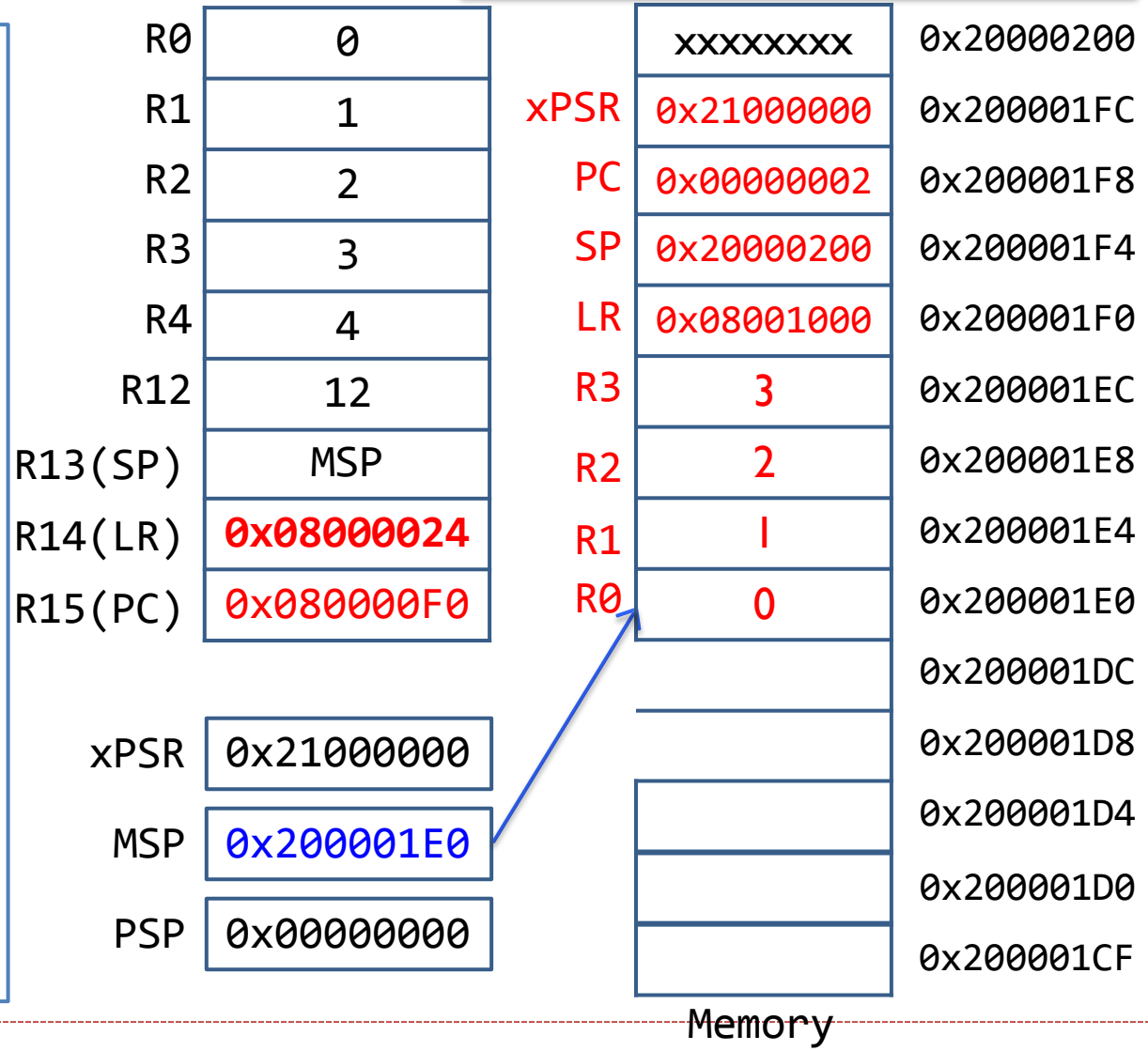


Interrupt: Stacking & Uns

UNSTACKING won't occur!

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1 0x0800001C
   BL  sine   0x08000020
   BX  lr     0x08000024
   ENDP
```

BL sine
Updates LR register



Interrupt: Stacking & Unstacking

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   PUSH {lr}
   ADD r4, #1
   BL  sine
   POP {lr}
   BX  lr
   ENDP
```

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0x08000024
R15(PC)	0x080000F0

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	xxxxxxxx	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x00000002	0x200001F8
SP	0x20000200	0x200001F4
LR	0x08001000	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
LR	0xFFFFFFF9	0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Fix the bug! Method 1

Interrupt: Stacking & Unstacking

```
__main PROC
   …          addr = 0x08000044
   MOV r3,#0
   …
   ENDP
              addr = 0x0800001C
SysTick_Handler PROC
   EXPORT SysTick_Handler
   PUSH {lr}
   ADD r4, #1
   BL  sine
   POP {PC}
   ENDP
```

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0x08000024
R15(PC)	0x080000F0

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	xxxxxxxx	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x00000002	0x200001F8
SP	0x20000200	0x200001F4
LR	0x08001000	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
LR	0xFFFFFFF9	0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Fix the bug! Method 2

Interrupt: Stacking & Unstacking

```
__main PROC
   …
   MOV r3,#0
   …
   ENDP

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1
   BL  sine
   LDR lr,=0xFFFFFFF9
   BX  lr
   ENDP
```

addr = 0x08000044

addr = 0x0800001C

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0x08000024
R15(PC)	0x080000F0

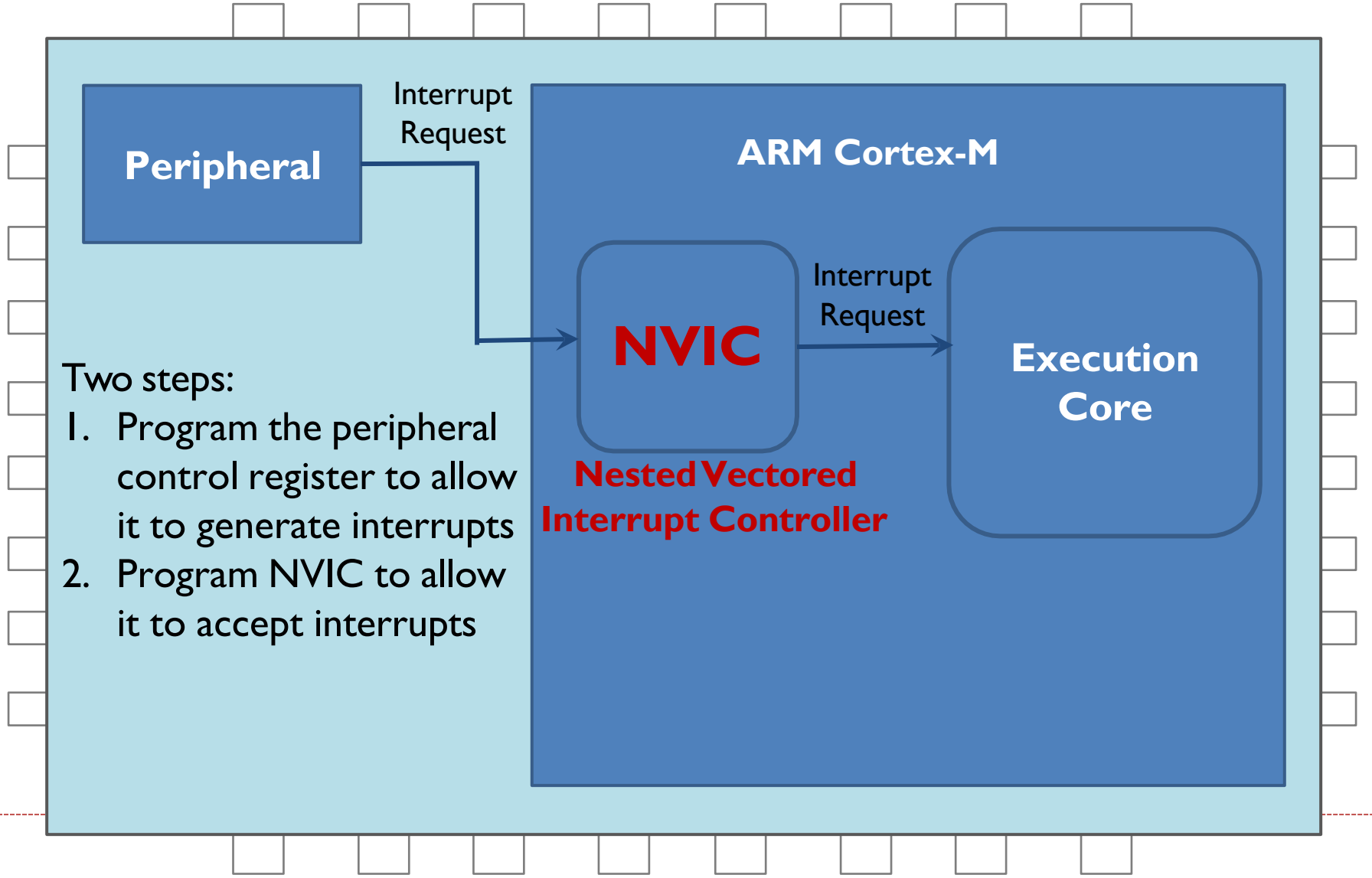
xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	xxxxxxxx	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x00000002	0x200001F8
SP	0x20000200	0x200001F4
LR	0x08001000	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

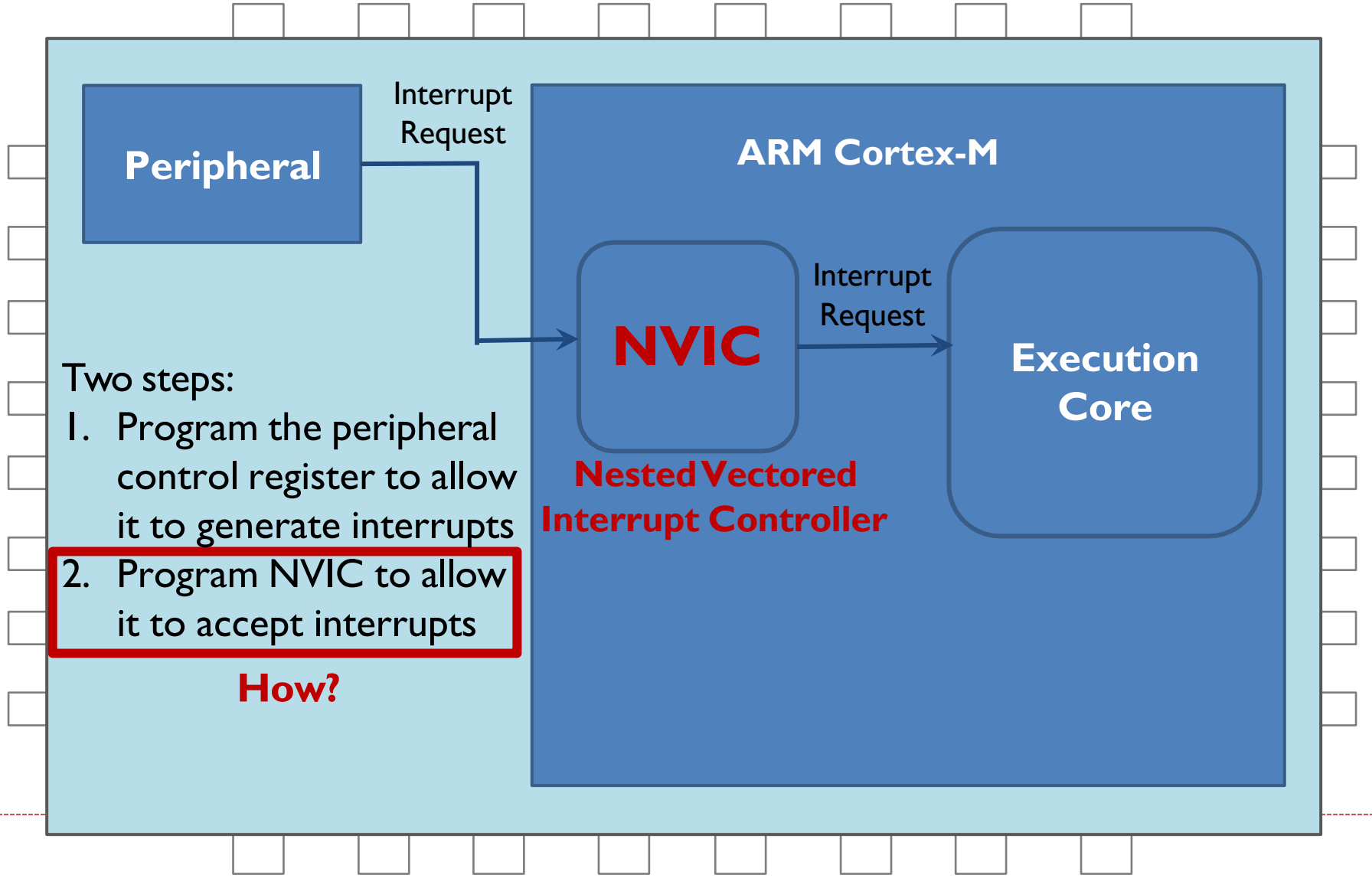
Memory

Fix the bug! Method 3 (not recommended)

Enable an Interrupt

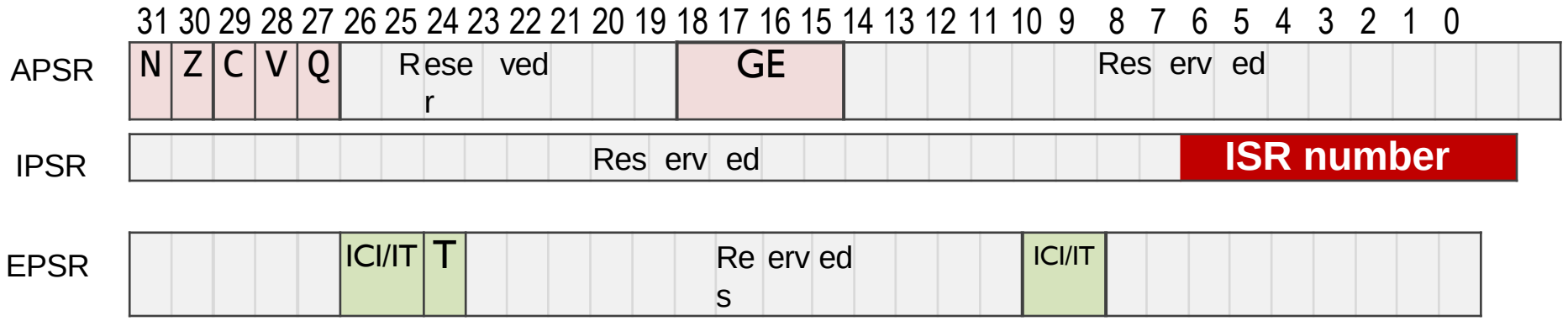


Enable an Interrupt



Interrupt Number in PSR

- Application PSR (**APSR**), Interrupt PSR (**IPSR**), Execution PSR (**EPSR**)

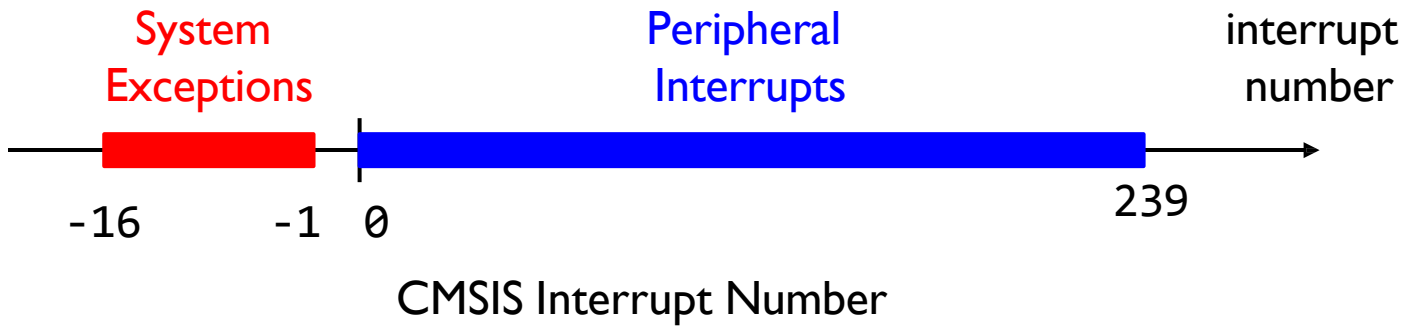


Combine them together into one register (**PSR**)

8-bit interrupt number in **PSR**
Range: **`0 - 255`**

Interrupt Number in CMSIS Library

- Interrupt number defined in ARM software library



- First 16 are system exceptions
  - CMSIS defines their interrupt numbers as negative
  - Defined by ARM core
- The rest 240 are peripheral interrupts
  - Peripheral interrupt number starts with 0.
  - Defined by chip manufacturers.

Interrupt Number in CMIS Library

```
/******  Cortex-M4 System Exceptions ********************************************************/
  NonMaskableInt_IRQn     = -14,     /* 2 Cortex-M4 Non Maskable Interrupt                    */
  HardFault_IRQn          = -13,     /* 3 Cortex-M4 Hard Fault Interrupt                      */
  MemoryManagement_IRQn   = -12,     /* 4 Cortex-M4 Memory Management Interrupt               */
  BusFault_IRQn           = -11,     /* 5 Cortex-M4 Bus Fault Interrupt                       */
  UsageFault_IRQn         = -10,     /* 6 Cortex-M4 Usage Fault Interrupt                     */
  SVCall_IRQn             = -5,      /* 11 Cortex-M4 SV Call Interrupt                        */
  DebugMonitor_IRQn       = -4,      /* 12 Cortex-M4 Debug Monitor Interrupt                  */
  PendSV_IRQn             = -2,      /* 14 Cortex-M4 Pend SV Interrupt                        */
  SysTick_IRQn            = -1,      /* 15 Cortex-M4 System Tick Interrupt                    */
/******  Peripheral Interrupt Numbers ********************************************************/
  WWDG_IRQn               = 0,       /* Window WatchDog Interrupt                             */
  PVD_PVM_IRQn            = 1,       /* PVD/PVM1,2,3,4 through EXTI Line detection Interrupts */
  TAMP_STAMP_IRQn         = 2,       /* Tamper and TimeStamp interrupts through the EXTI line */
  RTC_WKUP_IRQn           = 3,       /* RTC Wakeup interrupt through the EXTI line            */
  FLASH_IRQn              = 4,       /* FLASH global Interrupt                                */
  RCC_IRQn                = 5,       /* RCC global Interrupt                                  */
  EXTI0_IRQn              = 6,       /* EXTI Line0 Interrupt                                  */
  ...
```

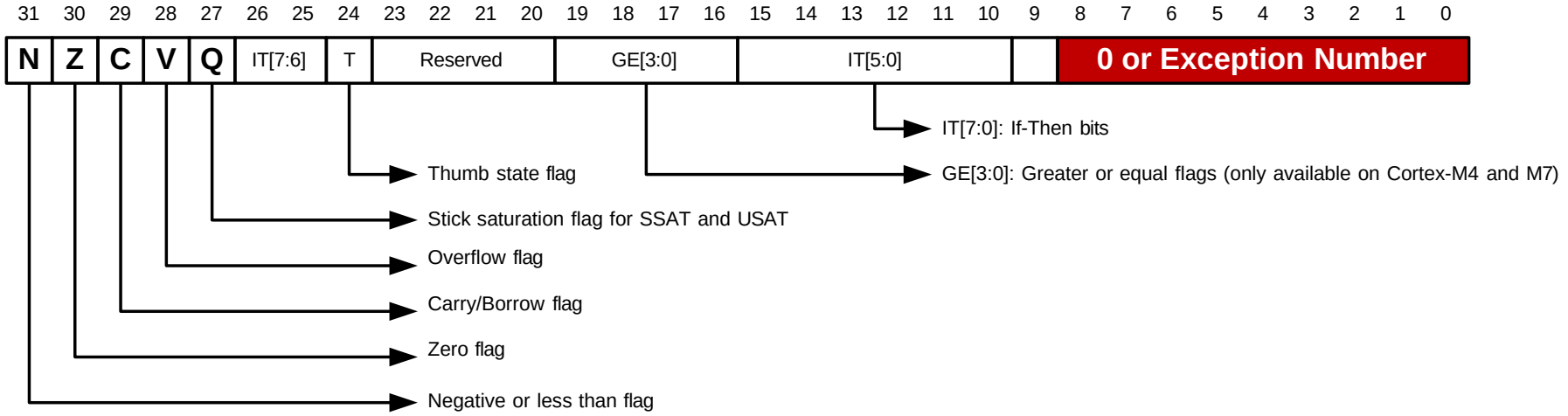
System Exceptions Defined by ARM

Peripheral Interrupts Defined by chip vendor

stm32l476xx.h

Interrupt Number in CMSIS *vs* in PSR

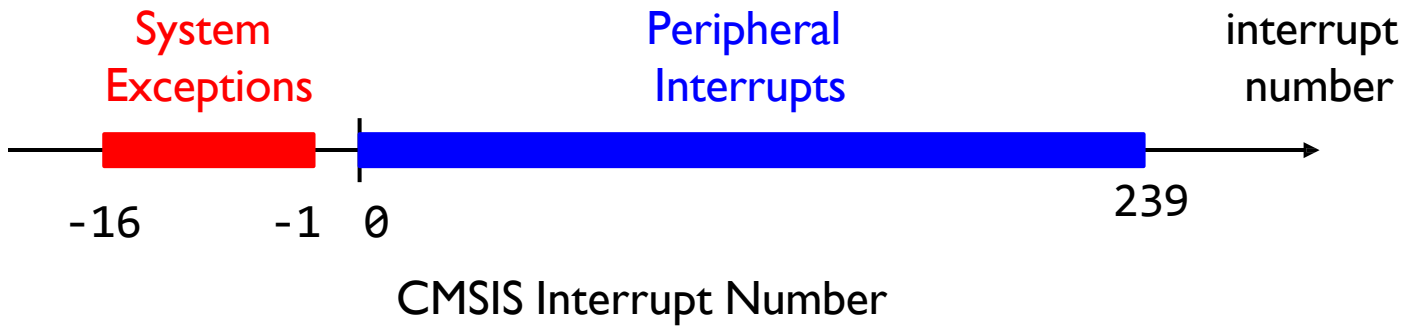
Interrupt number in Program Status Register (PSR)



Interrupt Number in PSR = 16 + Interrupt Number for CMSIS

Interrupt Number

- Interrupt number defined in ARM software library



Interrupt number for CMSIS functions

```
NVIC_DisableIRQ (IRQn);              // Disable interrupt
NVIC_EnableIRQ (IRQn);               // Enable interrupt
NVIC_ClearingPending (IRQn);         // clear pending status
NVIC_SetPriority (IRQn, priority); // set priority level
```

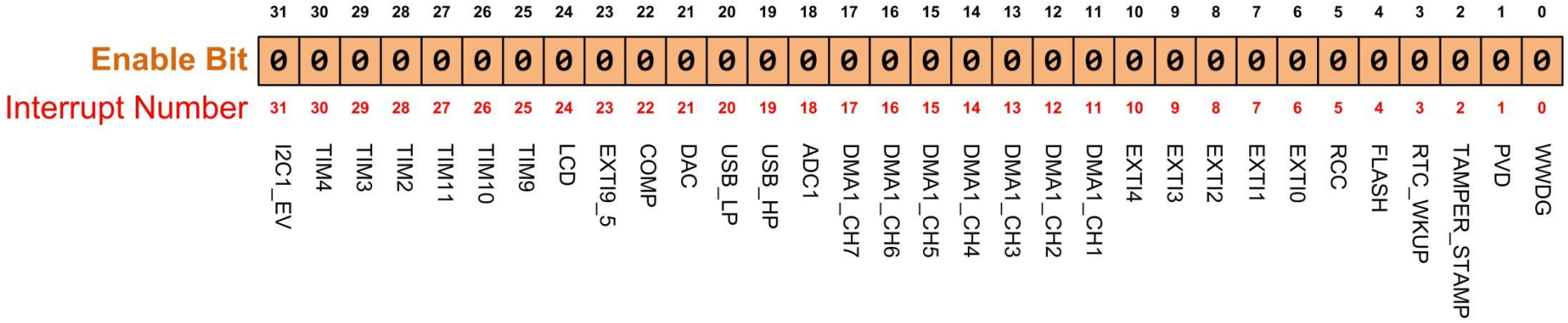
Enable an Interrupt

- Enable a system exception
  - Some are always enabled (cannot be disabled)
  - No centralized registers for enabling/disabling
  - Each are control by its corresponding components, such as SysTick module

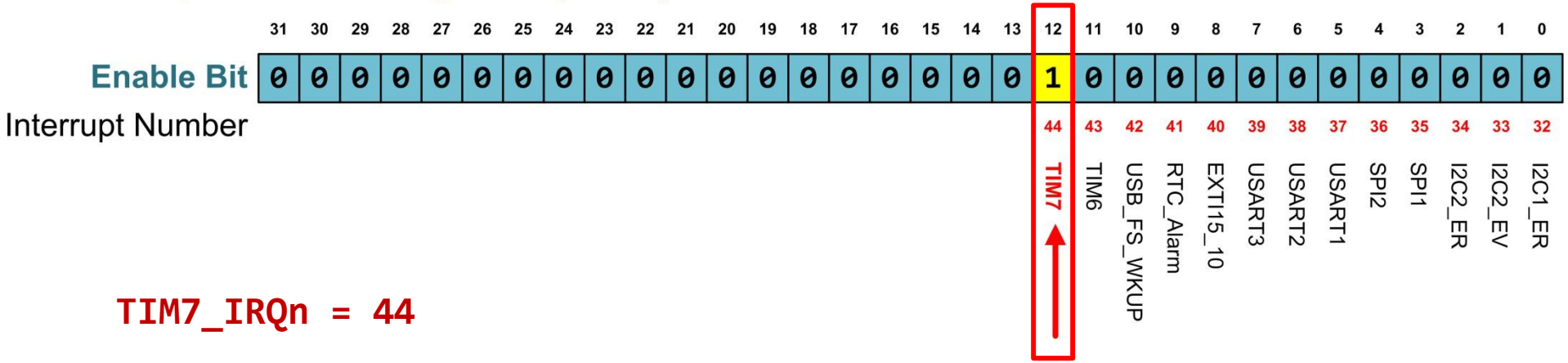
- Enable a peripheral interrupt
  - Centralized register arrays for enabling/disabling
  - **ISER** registers for enabling
  - **ICER** registers for disabling

Enabling Peripheral Interrupts

Interrupt Set Enable Register 0 (ISER0)



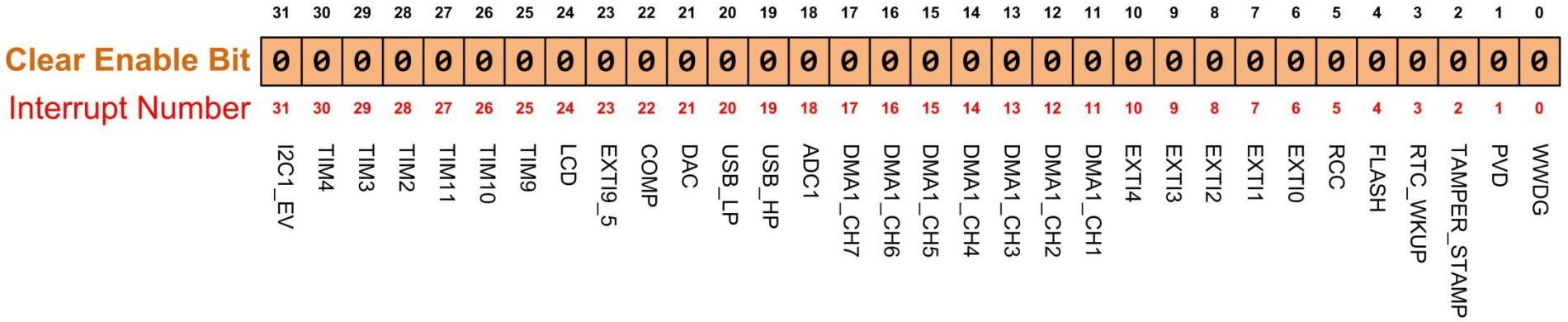
Interrupt Set Enable Register 1 (ISER1) *Address of ISER1 = Address of ISER0 + 4*



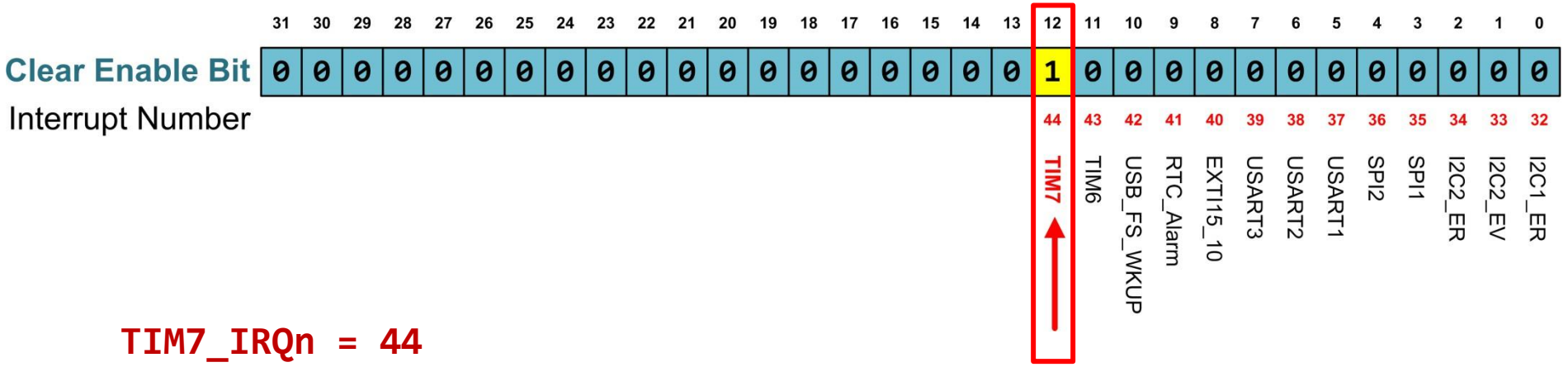

```
TIM7_IRQn = 44

NVIC->ISER[1] = 1 << 12;      // Enable Timer 7 interrupt
```

Disabling Peripheral Interrupts

Interrupt Clear Enable Register 0 (ICER0)



Interrupt Clear Enable Register 1 (ICER1) *Address of ICER1 = Address of ISER0 + 4*




```
TIM7_IRQn = 44

NVIC->ICER[1] = 1 << 12;      // Diable Timer 7 interrupt
```

Disable/Enable Peripheral Interrupts

- For all peripheral interrupts: `IRQn ≥ 0`
- Method 1:
  - `NVIC_EnableIRQ (IRQn);  // Enable interrupt`
  - `NVIC_DisableIRQ (IRQn); // Disable interrupt`
- Method 2:
  - Enable:
    - `NVIC->ISER[ IRQn / 32] = 1 << (IRQn % 32);`
    - Better solution:
    - `NVIC->ISER[ IRQn >> 5] = 1 << (IRQn & 0x1F);`
  - Disable:
    - `NVIC->ICER[ IRQn >> 5] = 1 << (IRQn & 0x1F);`

Interrupt Priority

- Inverse Relationship:
  - Lower priority value means higher urgency.
    - Priority of Interrupt A = 5,
    - Priority of Interrupt B = 2,
    - B has a higher priority/urgency than A.

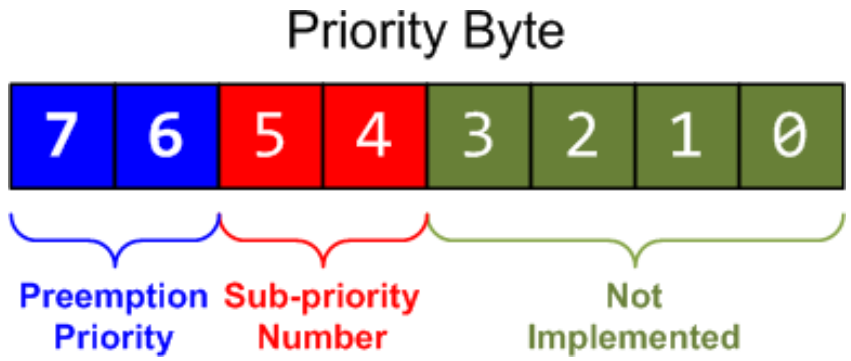
- Fixed priority for Reset, HardFault, and NMI.

Exception	IRQn	Priority
Reset	N/A	-3 (the highest)
Non-maskable Interrupt (NMI)	-14	-2 (2nd highest)
Hard Fault	-13	-1

- Adjustable for all the other interrupts

Interrupt Priority

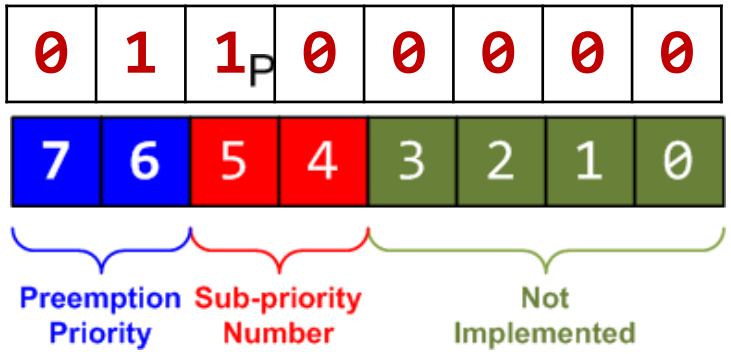
- Interrupt priority is configured by Interrupt Priority Register (IP)
- Each priority consists of two fields, including preempt priority number and sub-priority number.
  - The preempt priority number defines the priority for preemption.
  - The sub-priority number determines the order when multiple interrupts are pending with the same preempt priority number.



default setting

Interrupt Priority Levels

```
NVIC_SetPriority(7, 6);
```



IP = 0x60 = 96

core_cm4.h or core_cm3.h

```
typedef struct {
  ...
  // Interrupt Priority Register
  volatile uint8_t IP[240];
  ...
} NVIC_Type;
```

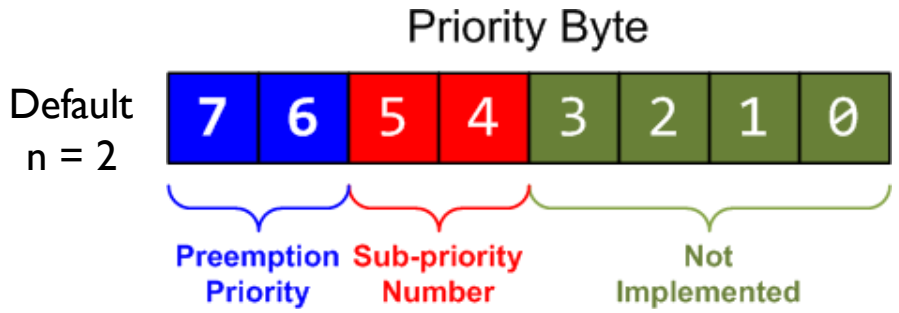
It is equivalent to:

```
NVIC->IP[7] = (6 << 4) & 0xff;
```

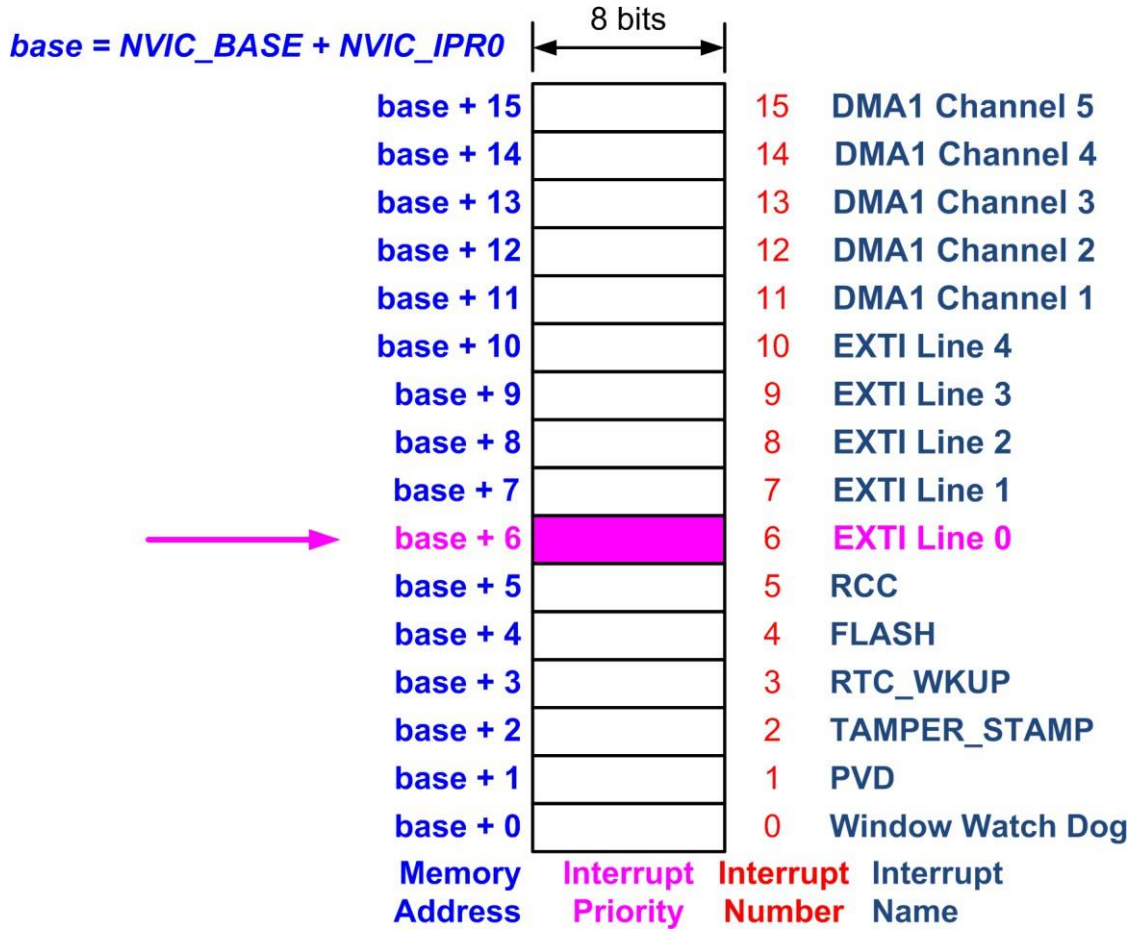
Preemption and Sub-priority Configuration

- `NVIC_SetPriorityGrouping(n)`
  - `Perform unlock, and update AIRCR register`

n	# of bits in preemption priority	# of bits in sub-priority
0	0	4
1	1	3
2 (default)	2	2
3	3	1
4	4	0

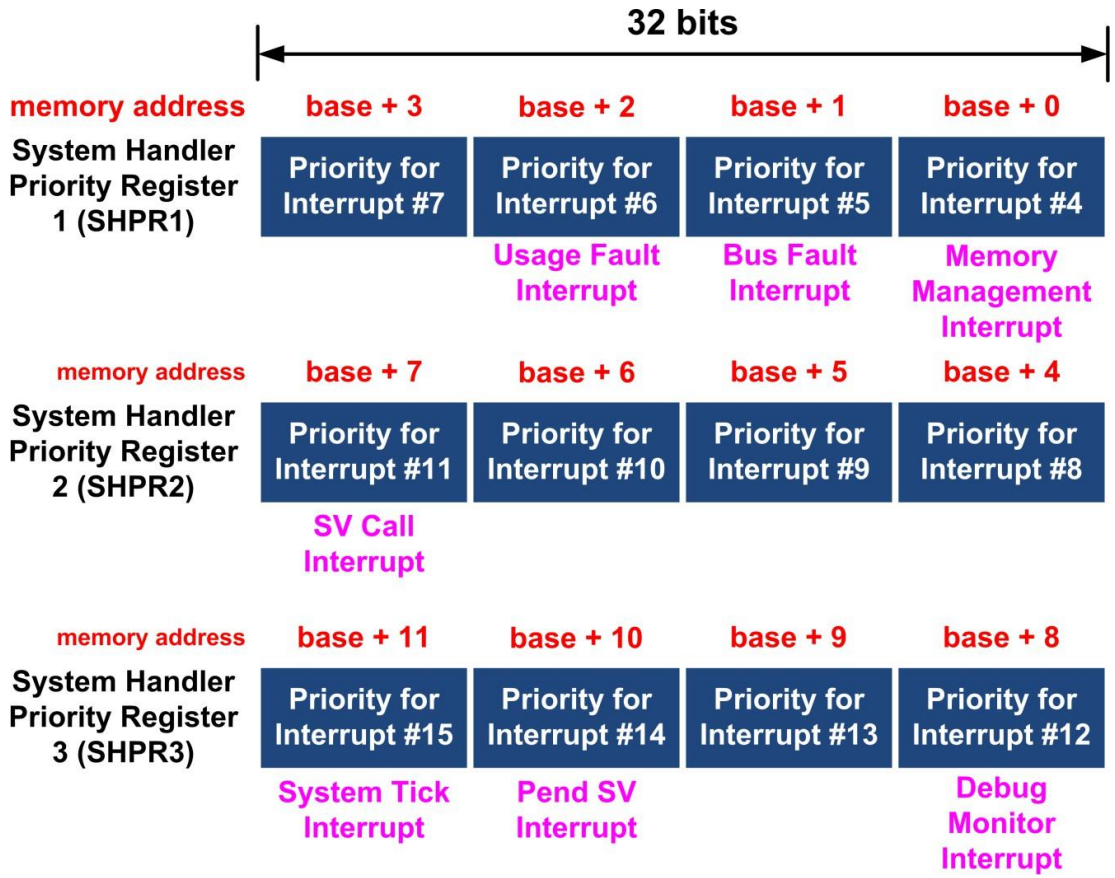


Priority of Peripheral Interrupts




```
// Set the priority for EXTI 0 (Interrupt number 6)
NVIC->IP[6] = 0xF0;
```

Priority of System Interrupts




```
// Set the priority of a system interrupt IRQn
SCB->SHP[(IRQn) & 0xF) - 4] = (priority << 4) & 0xFF;
```

Masking Priority

- Disable all interrupts with less urgency
  - For critical code, only certain interrupts are allowed.
  - BASEPRI: Disable all interrupts of specific priority level or higher priority level

```
// Disable interrupts with priority value same or higher
__set_BASEPRI( 5 << 4 )

// Critical code
...

// Remove BASEPRI masking
__set_BASEPRI(0);
```

Exception-masking registers (PRIMASK, FAULTMASK and BASEPRI)

- PRIMASK: Used to disable all exceptions except Non-maskable interrupt (NMI) and hard fault.
  - Write 1 to PRIMASK to disable all interrupts except NMI

```
MOV R0, #1
MSR PRIMASK, R0
```

  - Write 0 to PRIMASK to enable all interrupts

```
MOV R0, #0
MSR PRIMASK, R0
```

- FAULTMASK: Like PRIMASK but change the current priority level to -1, so that even hard fault handler is blocked

- BASEPRI: Disable interrupts only with priority lower than a certain level
  - Example, disable all exceptions with priority level higher than 0x60

```
MOV R0, #0x60
MSR BASEPRI, R0
```